BACSE202 - Database Systems

Entity Relationship Model: Types of Attributes, Relationship Types and Sets

Dr. Somasundaram R

Assistant Professor Sr. Grade 2

Emp Id: 23770

Department of IoT

School of Computer Science and Engineering (SCOPE)

Why the ER Model?

We want to develop a Library Management System. Should you directly create tables?

“What should we identify first?”

Books | **Students** | **Librarians**

Before creating database tables, we first identify real-world objects and how they relate to each other. This design approach is called the Entity Relationship (ER) Model.

INTERESTING FACT

The ER Model was proposed by Peter Chen in 1976, and remains one of the most widely used techniques for database design today.

What is an Entity?

Definition: An Entity is a real-world object that can be identified independently and about which data is stored.

EXAMPLES

- Student
- Faculty
- Course
- Department
- Book
- Employee

COLLEGE EXAMPLE

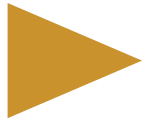
Entity	Description
Student	A student studying at VIT
Faculty	A teaching faculty member
Course	A subject offered by the university

Entity Set

Definition: An Entity Set is a collection of similar entities.

ENTITY

Student Arun



ENTITY SET

All Students

Student_ID	Name
101	Arun
102	Priya
103	Rahul

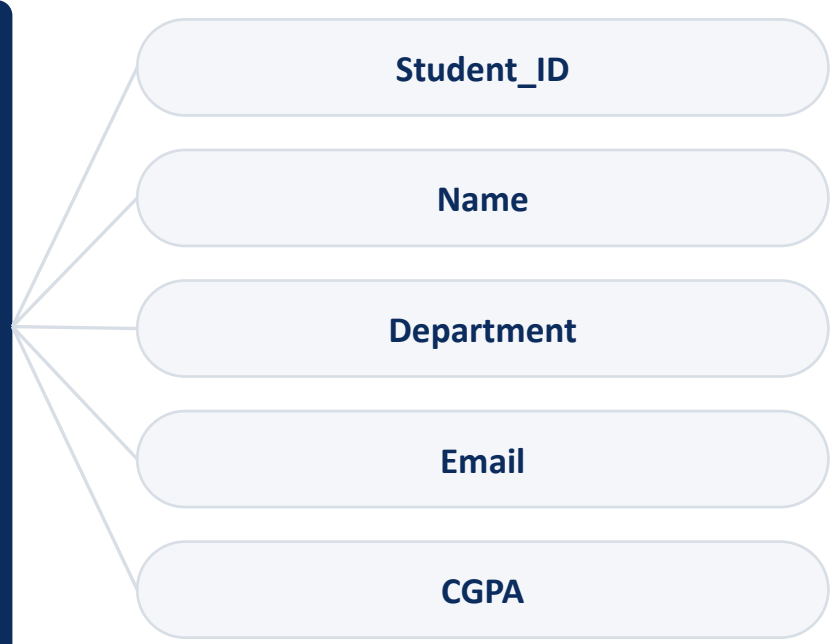
Student Arun ⟶ Entity **All students together** ⟶ Entity Set

Attributes

Definition: An Attribute describes the properties of an entity.

ENTITY

Student

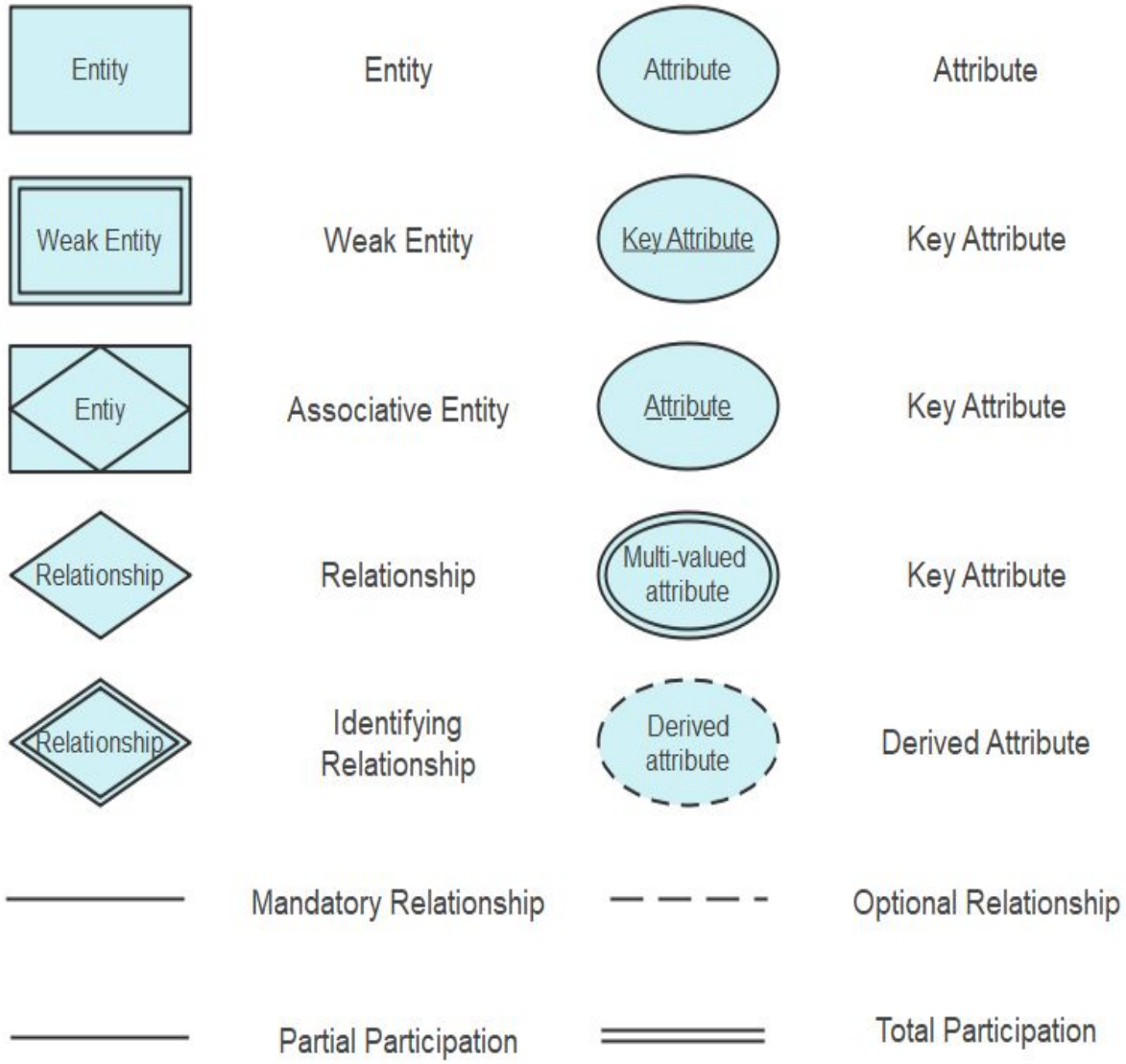


SIMPLE ANALOGY

Entity = Person

Attributes = Name, Age, Address

ER model Notations



Types of Attributes

Simple

Cannot be divided

Gender, Salary

Composite

Can be divided into parts

Name → First, Middle, Last
Address → Street, City, PIN

Single-Valued

Only one value

Date of Birth

Multi-Valued

Multiple values allowed

Phone Numbers:
9876543210, 9123456789

Derived

Calculated from another attribute

DOB → Age
CGPA → Percentage

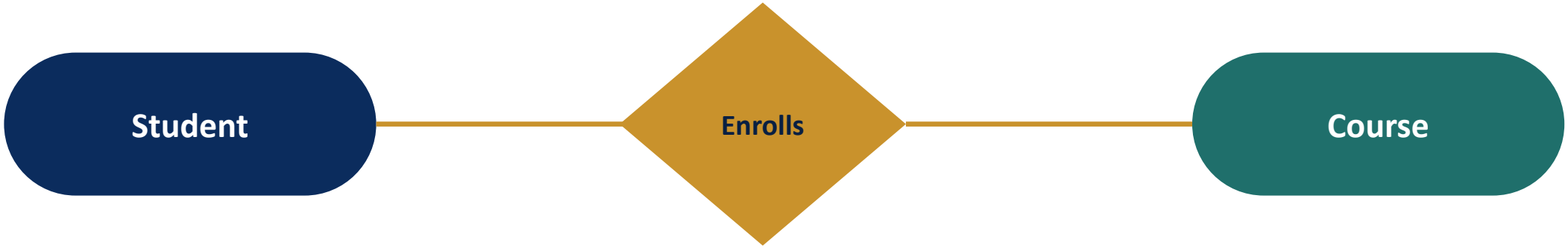
Stored

Stored directly in the database

Date of Birth

Relationship

Definition: A Relationship represents an association between two or more entities.

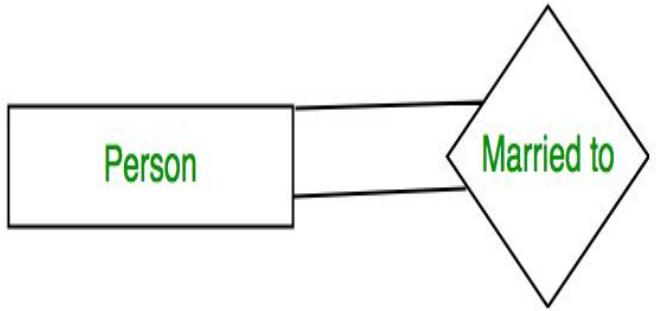


OTHER EXAMPLES

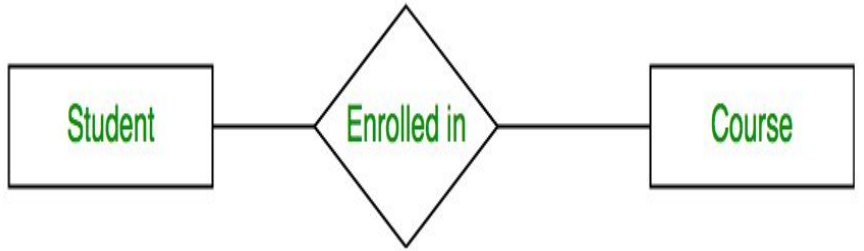
Faculty teaches Course	Customer places Order	Doctor treats Patient

Degree of Relationship

1. Unary Relationship
   - Relationship among one entity set.

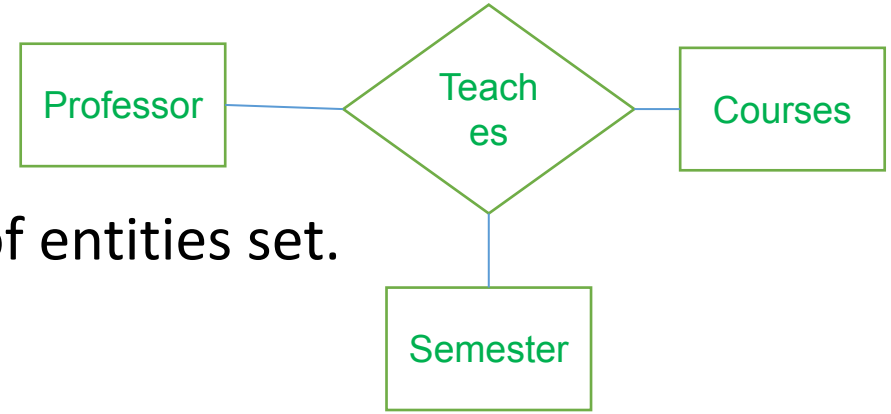


2. Binary Relationship
   - Relationship among two entities set.



3. Ternary Relationship
   - Relationship among 3 entities set.

4. N-ary Relationship :
   - Relationship between n number of entities set.



Relationship Types

One-to-One (1:1)

One HOD manages one Department.

One-to-Many (1:N)

One Department has many Students.

Many-to-One (N:1)

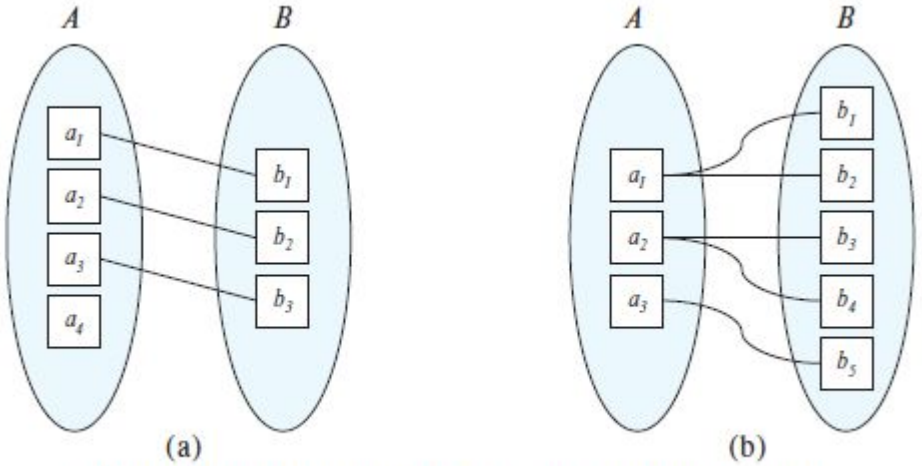
Many Students belong to one Department.

Many-to-Many (M:N)

Many Students enroll in many Courses.

Many-to-Many (M:N) is the most common relationship type in university databases.

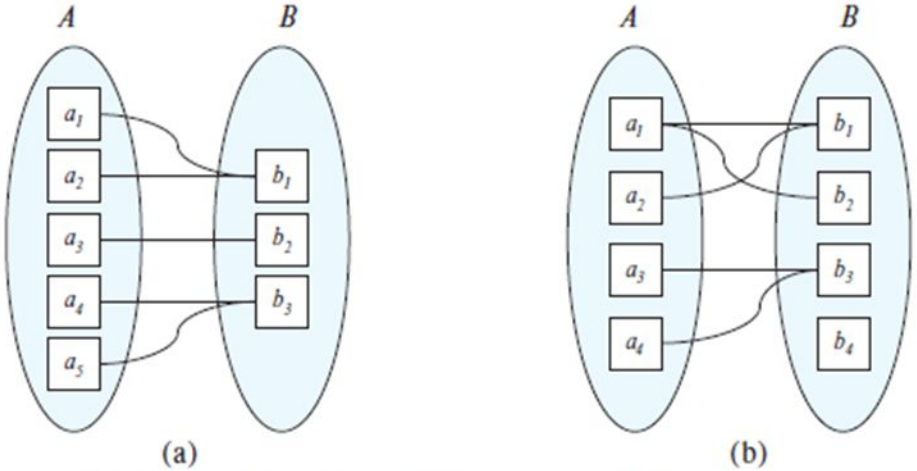
Mapping Cardinalities



Mapping cardinalities. (a) One-to-one. (b) One-to-many.

One-to-one. An entity in A is associated with **at most one** entity in B, and an entity in B is associated with at most one entity in A.

One-to-many. An entity in A is associated with **any number** (zero or more) of entities in B. An entity in B, however, can be associated with at most one entity in A.



Mapping cardinalities. (a) Many-to-one. (b) Many-to-many.

Many-to-one. An entity in A is associated with **at most one entity** in B. An entity in B, however, can be associated with **any number** (zero or more) of entities in A.

Many-to-many. An entity in *A* is associated with **any number** (zero or more) of entities in *B*, and an entity in *B* is associated with **any number** (zero or more) of entities in *A*.

Mapping Cardinalities

Identify the mapping Cardinalities

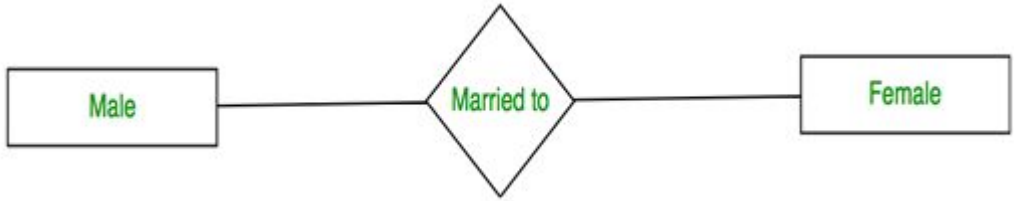
- One to One



- One to Many



- Many to Many



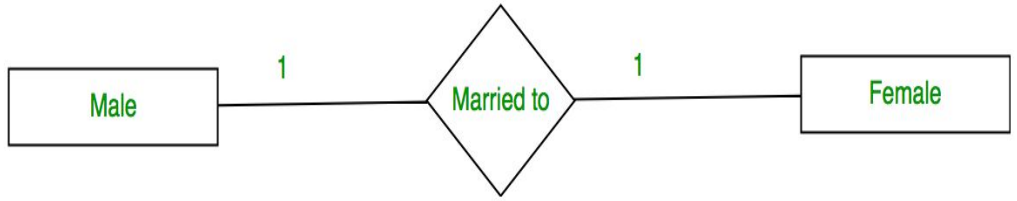
- Many to One



Mapping Cardinalities

Identify the mapping Cardinalities

- One to One



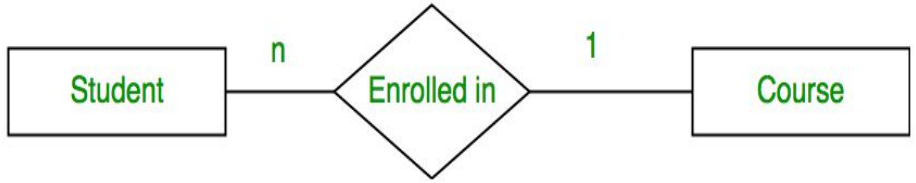
- One to Many



- Many to Many



- Many to One



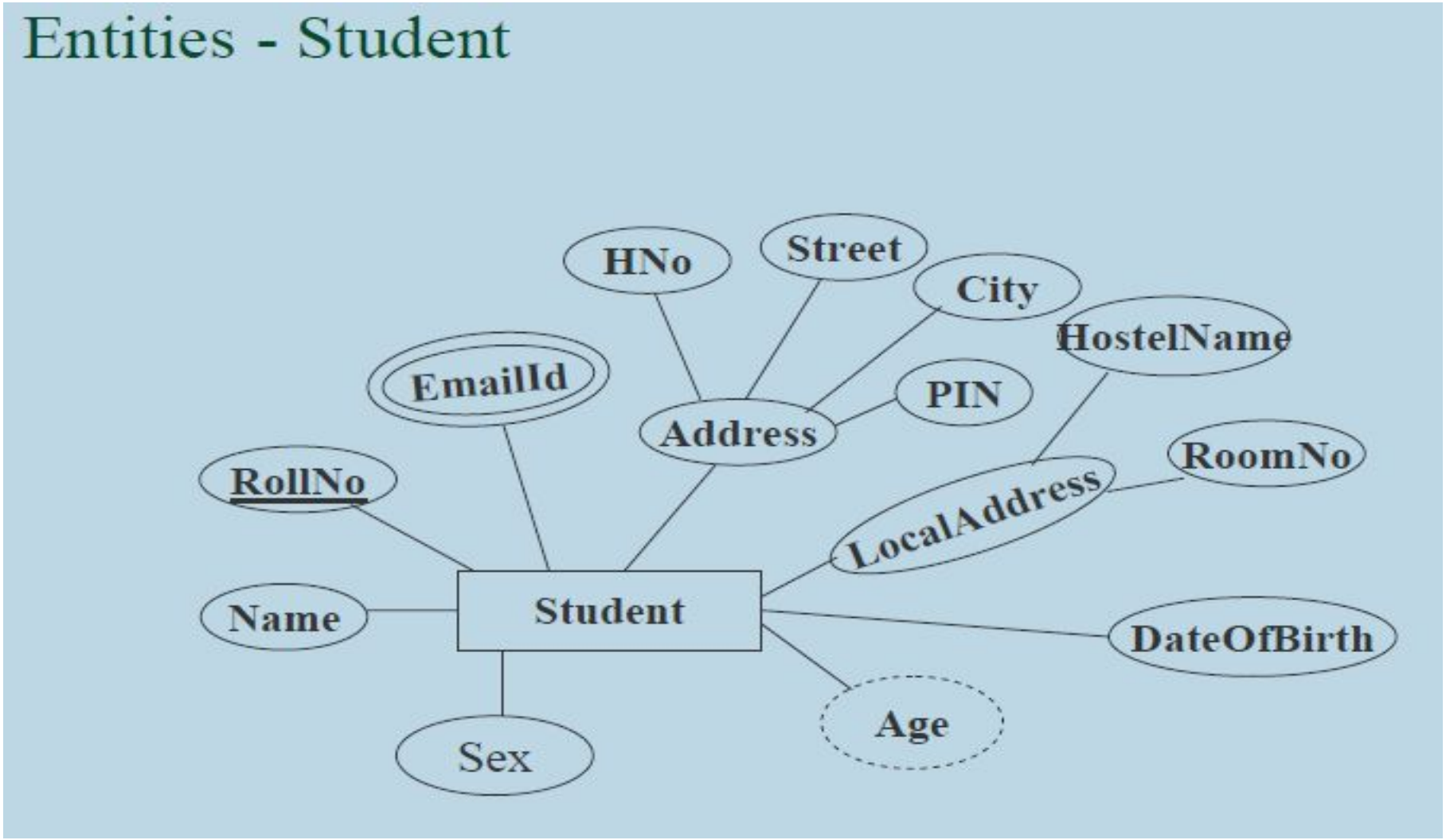
Relationship Set

Definition: A Relationship Set is a collection of similar relationships.

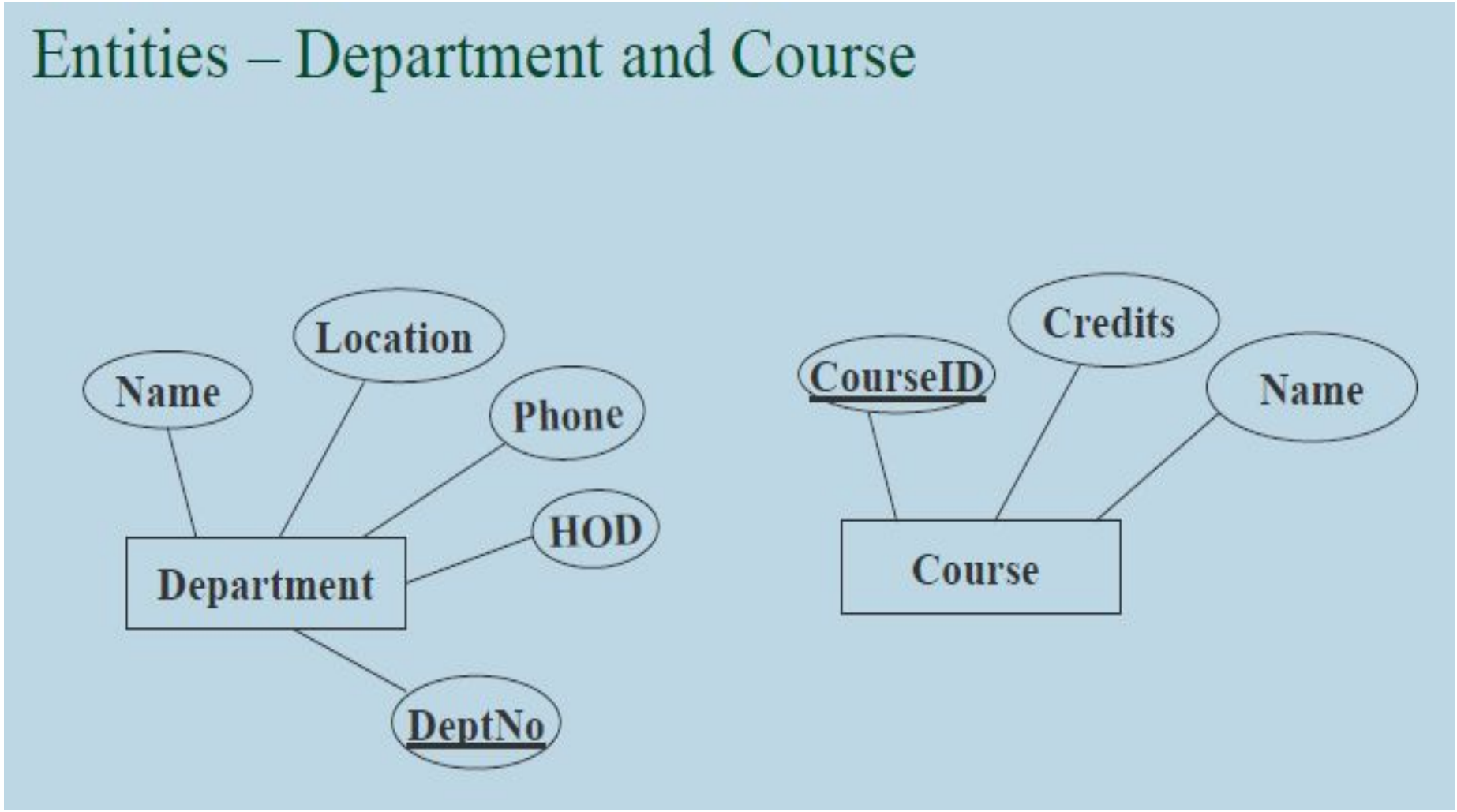
Student	Course
Arun	DBMS
Arun	Python
Priya	DBMS

All these “Enrolls” relationships together form the Relationship Set.

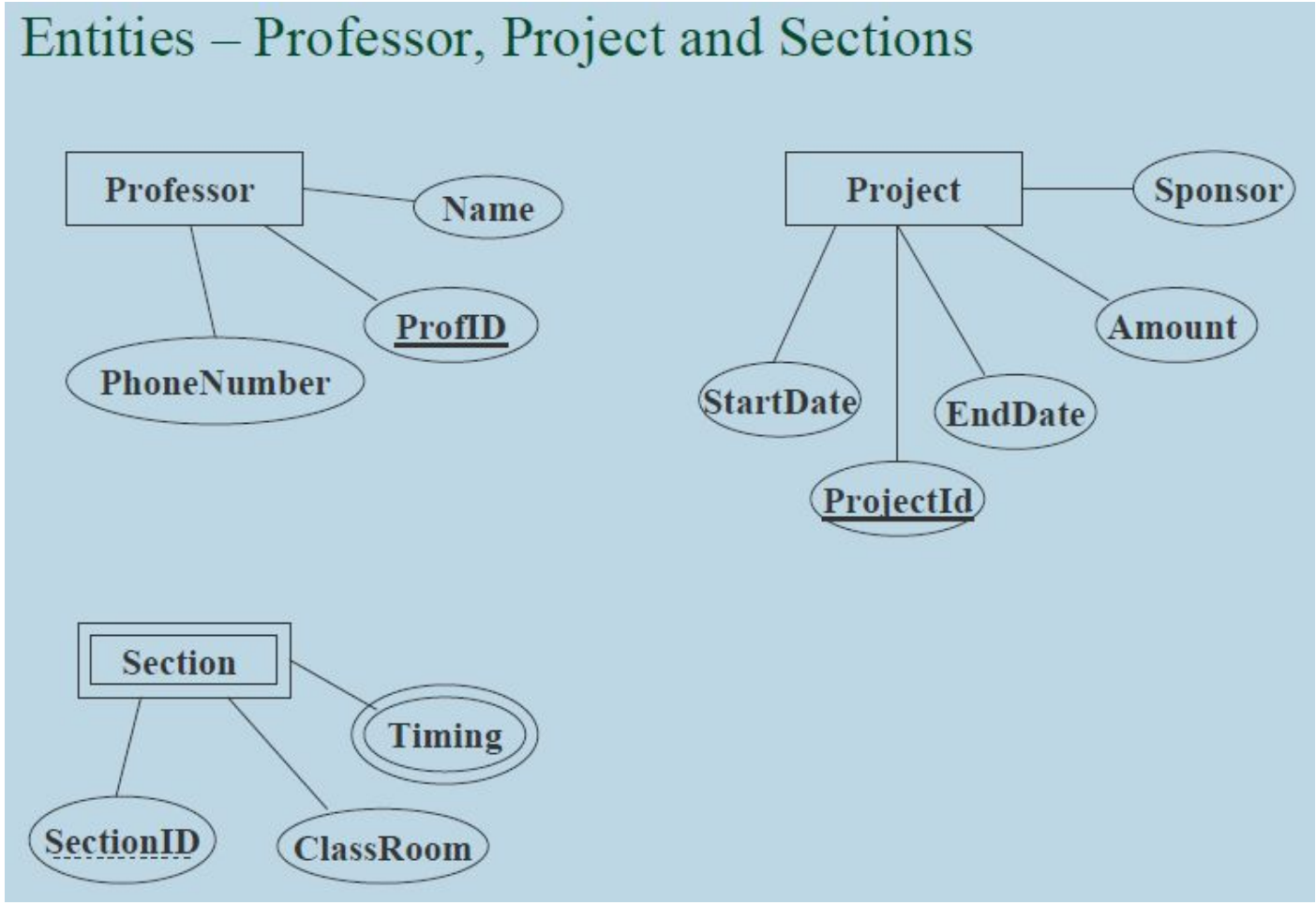
ER Diagram construction



ER Diagram


Entities – Department and Course
Location
Name
Phone
HOD
Department
DeptNo
Credits
CourseID
Name
Course

ER Diagram


Entities – Professor, Project and Sections
Professor
Name
ProfID
PhoneNumber
Project
Sponsor
Amount
StartDate
EndDate
ProjectId
Section
Timing
SectionID
ClassRoom

Thank You

BACSE202 - Database Systems

ER-to-Relational Mapping : From Entity Relationship Diagrams to Relational Schemas

Dr. Somasundaram R

Assistant Professor Sr. Grade 2

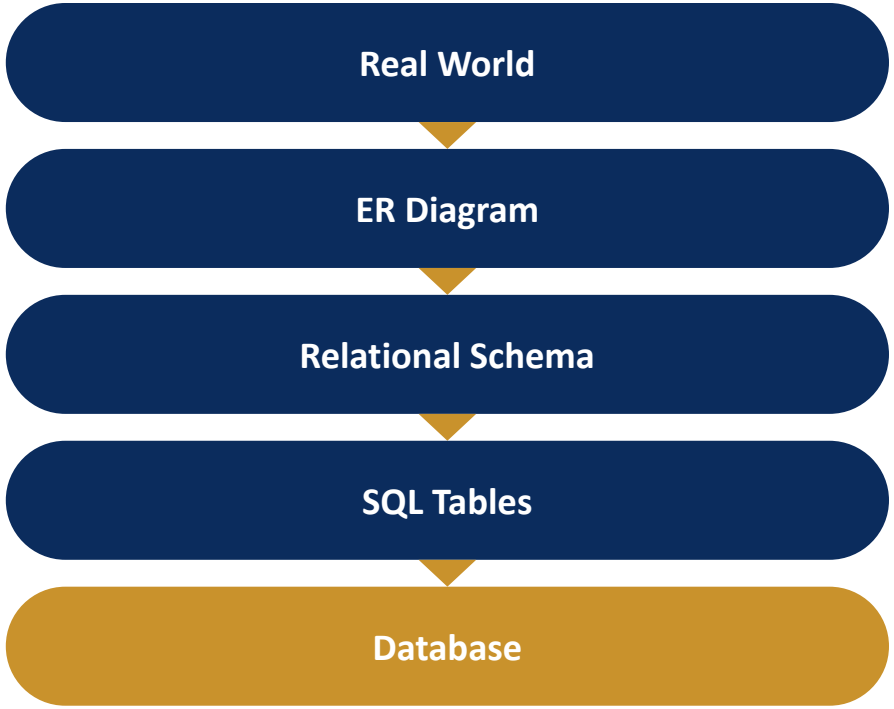
Emp Id: 23770

Department of IoT

School of Computer Science and Engineering (SCOPE)

Why Map the ER Model to a Relational Schema?

The ER Model is used to design a database, whereas a Relational Database stores data in the form of tables. Before implementation, the ER model must be converted into a relational schema.



KEY IDEA

Mapping bridges conceptual design (ER diagram) and physical implementation (SQL tables) — every entity, attribute, and relationship must find a home in a table.

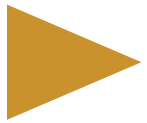
Mapping a Strong Entity

- Each Strong Entity becomes one relation (table)
- The entity name becomes the table name
- The attributes become columns
- The key attribute becomes the Primary Key

EXAMPLE

ENTITY: STUDENT

```
Student_ID
Name
Department
```



Student_ID (PK)	Name	Department
101	Arun	CSE

Learning Outcomes

After completing this lecture, students will be able to:

1. Explain the need for ER-to-Relational Mapping
2. Convert Strong Entities into relations
3. Convert Weak Entities into relations
4. Map different relationship types into tables
5. Design a relational schema from an ER diagram

Mapping a Weak Entity

Rule: Each Weak Entity also becomes a relation. The relation contains the attributes of the Weak Entity plus the Primary Key of the Owner Entity (as a Foreign Key).
Primary Key: Owner Key + Partial Key

EXAMPLE



Employee_ID (FK)	Dependent_Name	Age
E101	**Ravi**	12

Primary Key = (Employee_ID, Dependent_Name)

Mapping a One-to-One Relationship (1:1)

Rule: Include the Primary Key of one entity as a Foreign Key in the other relation.



RESULTING RELATION

Person_ID (PK)	Name	Passport_No (FK)
P01	Arun	**N1234567**

Mapping a One-to-Many Relationship (1:N)

Rule: Place the Primary Key of the One side into the Many side as a Foreign Key.

DEPARTMENT (One)

Dept_ID (PK)	Dept_Name
D01	CSE



STUDENT (Many)

Student_ID (PK)	Name	Dept_ID (FK)
101	Arun	**D01**

One Department → Many Students (Dept_ID travels to the STUDENT table)

Mapping a Many-to-Many Relationship (M:N)

- Create a new relation
- It contains the Primary Key of Entity 1 and Entity 2 — both become Foreign Keys
- Together they form the Composite Primary Key



Student_ID (FK)	Course_ID (FK)
101	C01

Primary Key = (Student_ID, Course_ID)

Mapping Multivalued Attributes

Rule: A multivalued attribute cannot be stored directly in one column — create a separate relation for it.

STUDENT

Student_ID
Phone_Number *



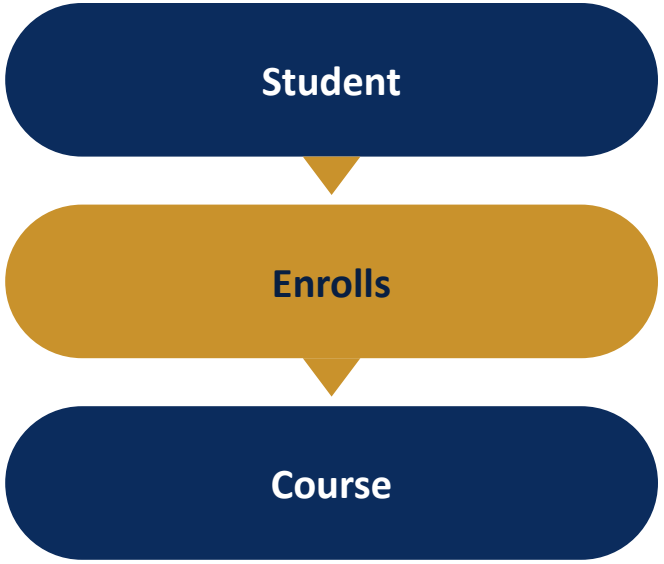
Student_ID (FK)	Phone_Number
101	9876543210
101	9123456789

Primary Key = (Student_ID, Phone_Number)

** asterisk denotes a multivalued attribute*

Complete Example — Student, Course, Enrolls

ER MODEL



RELATIONAL SCHEMA

STUDENT

`(Student_ID, Name)`

COURSE

`(Course_ID, Course_Name)`

ENROLLMENT

`(Student_ID, Course_ID)`

Steps for ER-to-Relational Mapping

1. Convert each Strong Entity into a relation
2. Convert each Weak Entity into a relation
3. Convert attributes into columns
4. Convert key attributes into Primary Keys
5. Map relationships using Foreign Keys
6. Create a new relation for Many-to-Many relationships
7. Handle multivalued attributes using separate relations

Hospital Management System

Entities: Patient, Doctor, Appointment. Convert the ER model into relational tables.

PATIENT

Patient_ID (PK)	Name	Age
P01	Meena	34

DOCTOR

Doctor_ID (PK)	Name	Specialization
D01	Dr. Rao	Cardiology

APPOINTMENT

Patient_ID (FK)	Doctor_ID (FK)	Date	Time
P01	**D01**	2026-07-27	10:30

Primary Keys, Foreign Keys, and how the Many-to-Many relationship between Patient and Doctor is resolved through APPOINTMENT.

ER-to-Relational Mapping

ER MODEL

Strong Entity	Table
Weak Entity	Table + Foreign Key
1 : 1 Relationship	Foreign Key
1 : N Relationship	Foreign Key on the Many side
M : N Relationship	New Relation

SUMMARY

- Mapping Strong Entity
- Mapping Weak Entity
- Mapping One-to-One Relationship
- Mapping One-to-Many Relationship
- Mapping Many-to-Many Relationship
- Mapping Multivalued Attributes
- Designing Relational Schemas

Thank You

BACSE202 - Database Systems

Extended ER Model: Generalization - Specialization – Aggregations Guidelines for Relational Schema

Dr. Somasundaram R

Assistant Professor Sr. Grade 2

Emp Id: 23770

Department of IoT

School of Computer Science and Engineering (SCOPE)

Extended ER Model

- EER is a high-level data model that incorporates the extensions to the original ER model.
- **It is a diagrammatic technique for displaying the following concepts**
  - Sub Class and Super Class
  - Specialization and Generalization
  - Union or Category
  - Aggregation
- These concepts are used to construct an Enhanced Entity-Relationship (EER) schema, and the resulting diagrams are called EER diagrams.

Sub Class and Super Class

- Sub class and Super class relationship leads the concept of Inheritance. The relationship between sub class and super class is denoted with symbol a **hollow arrow-head pointing from the specialized entity to the other entity**

Super Class

- **Super Class :** Super class is an entity type that has a relationship with one or more subtypes.
- An entity cannot exist in database merely by being member of any super class.

For example: **Shape** super class is having sub groups as **Square, Circle, Triangle**.

Sub Class

- **Sub Class :** Sub class is a group of entities with unique attributes.
- Sub class inherit properties and attributes from its super class.

For example: Square, Circle, Triangle are the sub class of Shape super class.

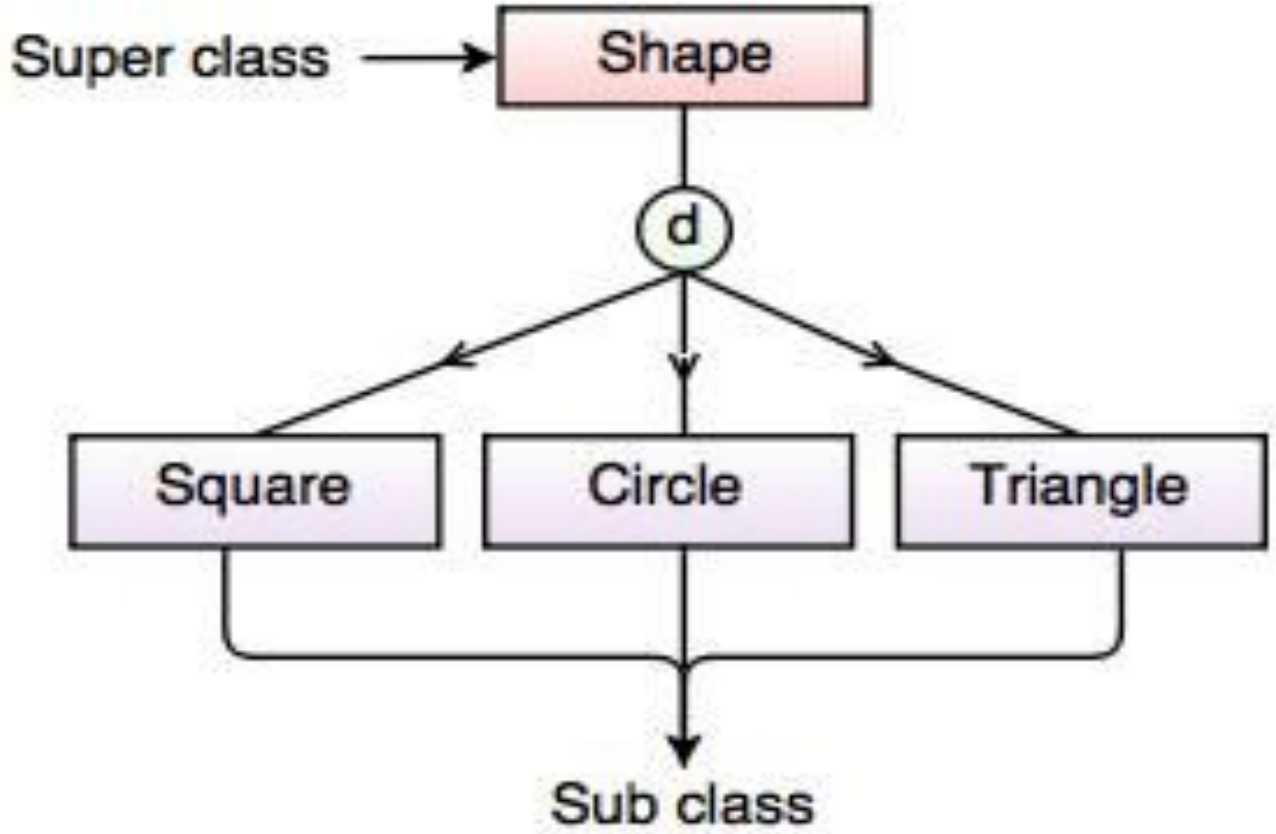
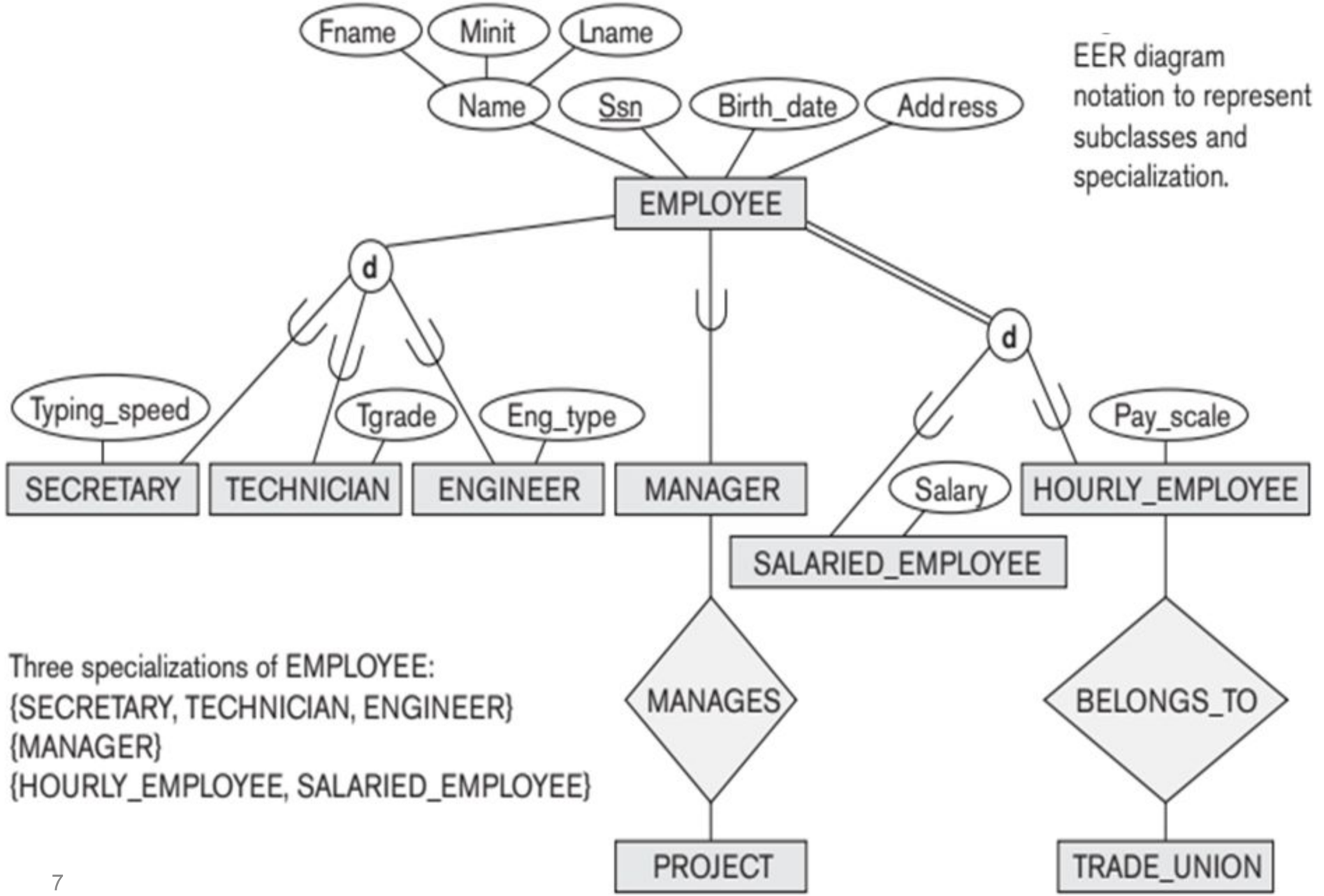


Fig. Super class/Sub class Relationship



EER diagram notation to represent subclasses and specialization.

Three specializations of EMPLOYEE:
{SECRETARY, TECHNICIAN, ENGINEER}
{MANAGER}
{HOURLY_EMPLOYEE, SALARIED_EMPLOYEE}

Constraints and its Types

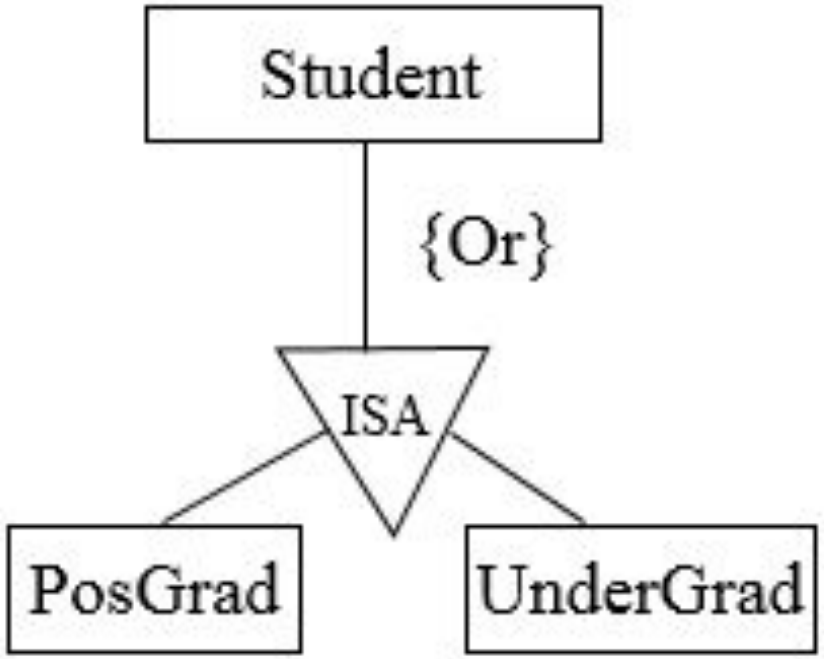
- There are three constraints that may apply to a specialization/generalization:
  - **Disjoint Constraints and**
  - **Completeness Constraints.**
  - **Membership Constraints**

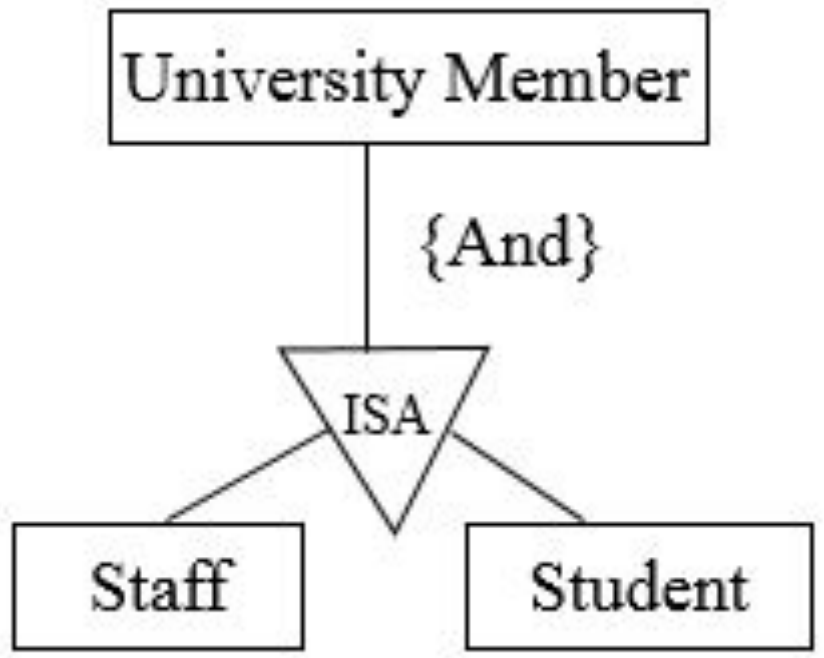
Disjointness Constraints on Specialization and Generalization

- There are two type of constraints in specializations.
- **Disjointness** specifies that the subclasses of the specialization must be disjoint.
- There are four further possible constraints on specialization
  - Disjoint total
  - Disjoint partial
  - Overlapping total
  - Overlapping partial
- **Completeness**
  - Decide whether a super type instance must also be a member of at least one subtype.

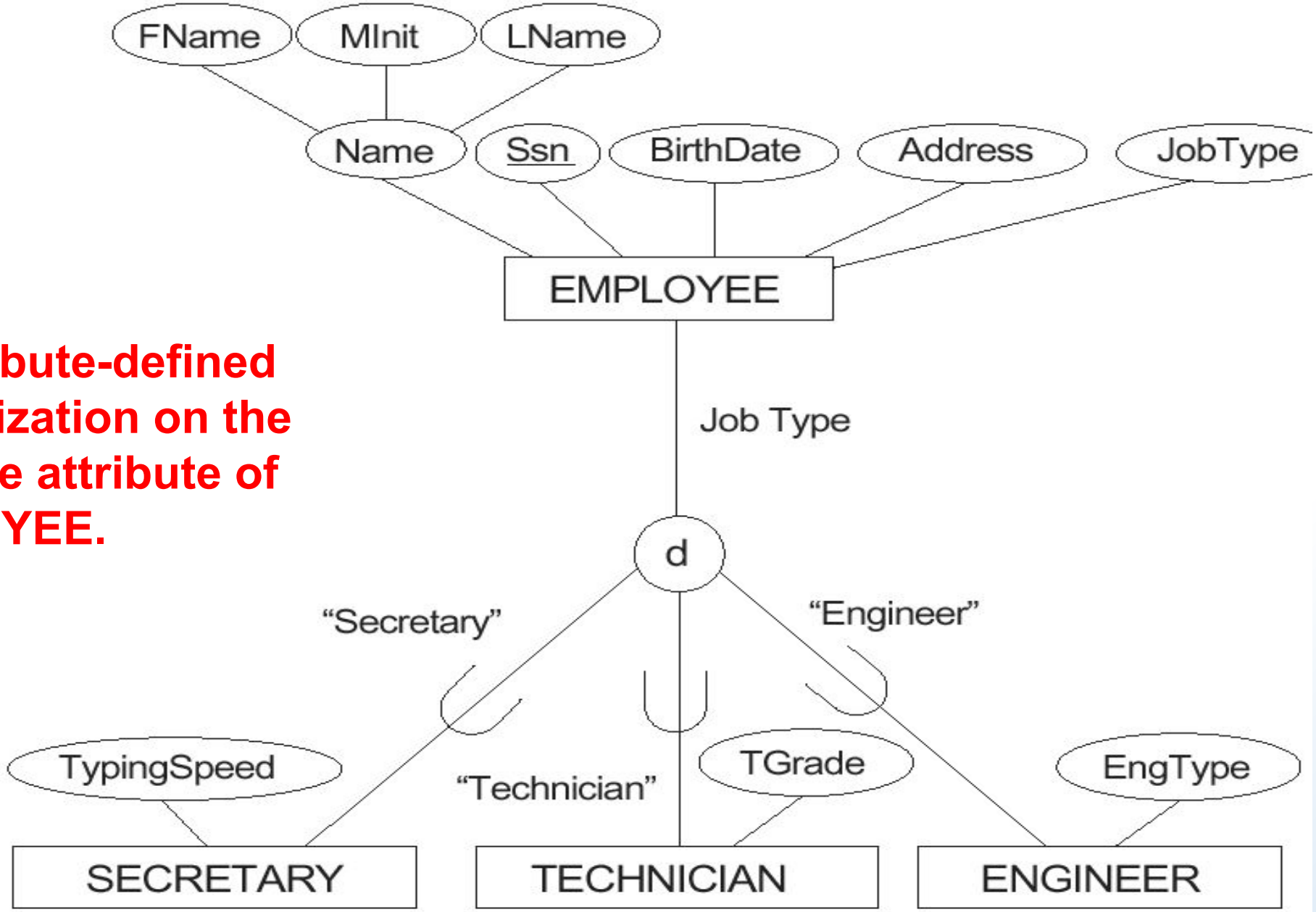
Disjoint Constraint

- **Disjoint (d)**
  - The subclasses must have disjoint sets of entities
- **Overlap (o)**
  - The subclasses may have overlapping sets of entities

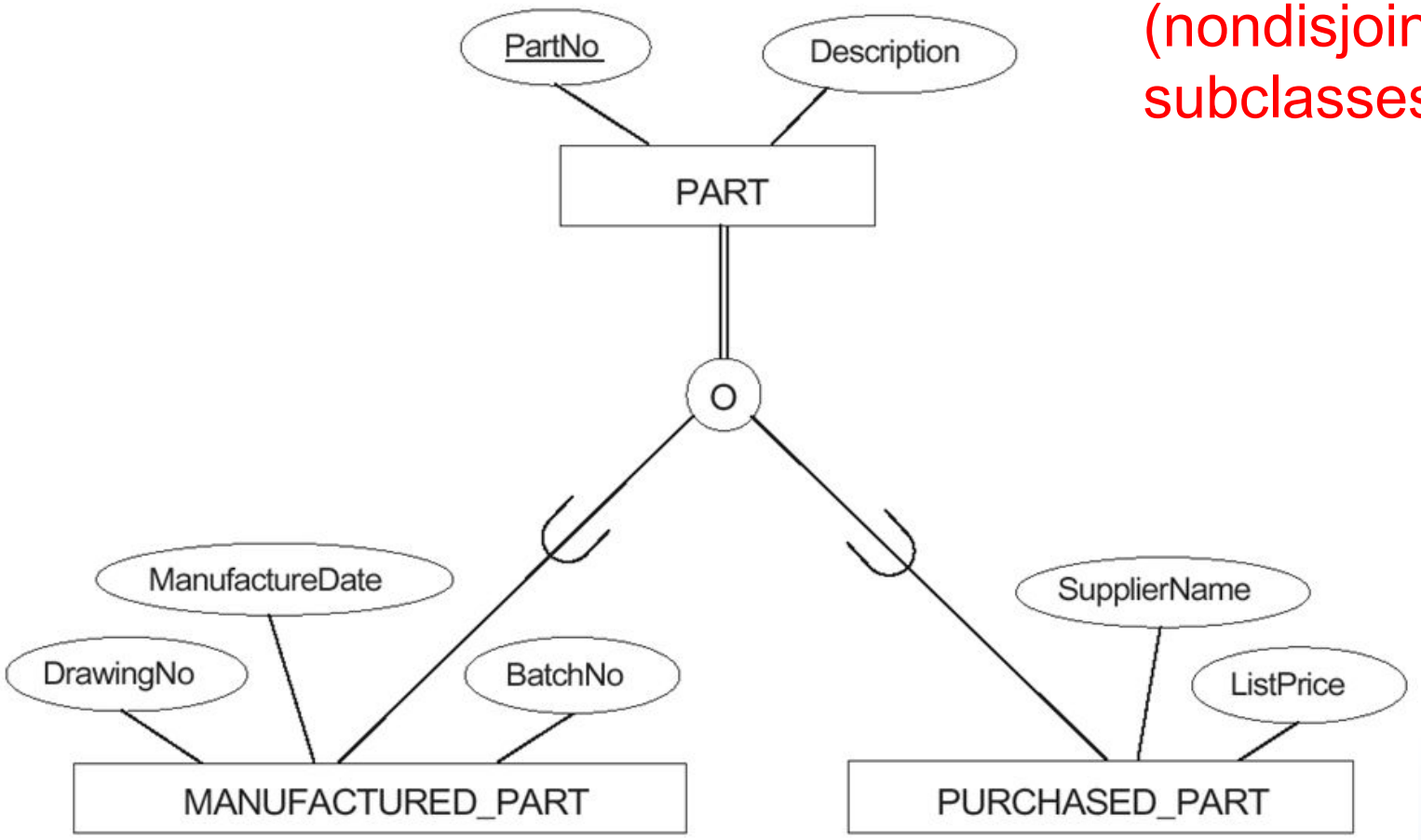




An attribute-defined specialization on the JobType attribute of EMPLOYEE.

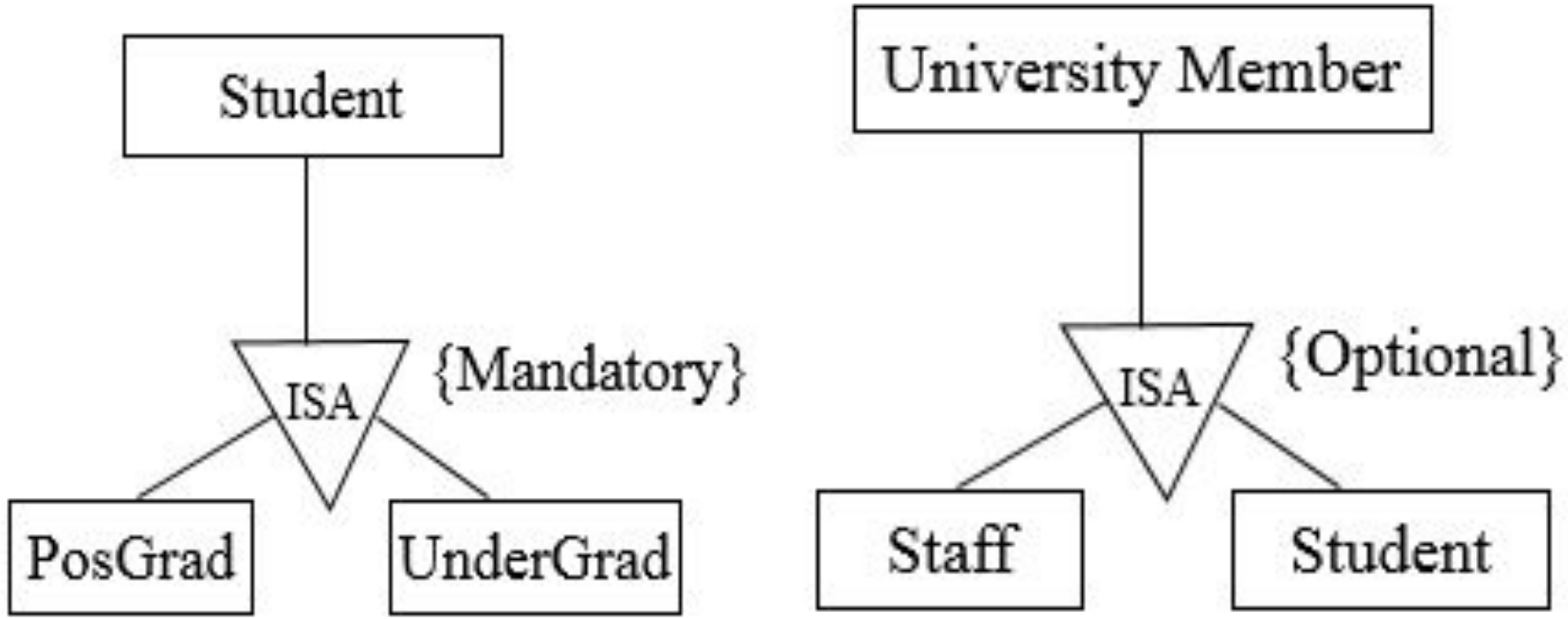


Notation for specialization with overlapping (nondisjoint) subclasses.



Completeness Constraints

- **Partial**
  - An entity may not belong to any of the subclasses (single-line)
- **Total**
  - Every entity in the superclass must be a member of some subclass (double-edge)



Total

Partial

(Specialization)

(Generalization)

Specialization

- An entity set may include subgroupings of entities that are distinct in some way from other entities in the set.
- **Specialization** is the process of defining a *set of subclasses* of an entity type. This entity type is called the **superclass** of the specialization.
- It is a **top down approach**, in which one higher entity can be broken down into two lower level entity.
- It maximizes the difference between the members of an entity by identifying the unique characteristic or attributes of each member.

- Specialization is depicted by a **hollow arrow-head pointing from the specialized entity to the other entity**

- The specialization of *person* allows us to distinguish among person entities according to whether they correspond to employees or students

- In general, a person could be an employee, a student, both, or neither.

- We refer to this relationship as the **ISA** relationship, which stands for "**is a**" and represents, for example, that an instructor "**is a**" employee.

- An entities B is said to be a subclass of another entity A. If and only if it shares an "**is-a**" relationship with A.
- Example: *Car "**is-a**" vehicle.*

  *Manager "**is-a**" Employee.*
- An entity of class B is said to be a **specialization** of entities of class A.
- Conversely, entities of class A are **generalization** of the class B entities.

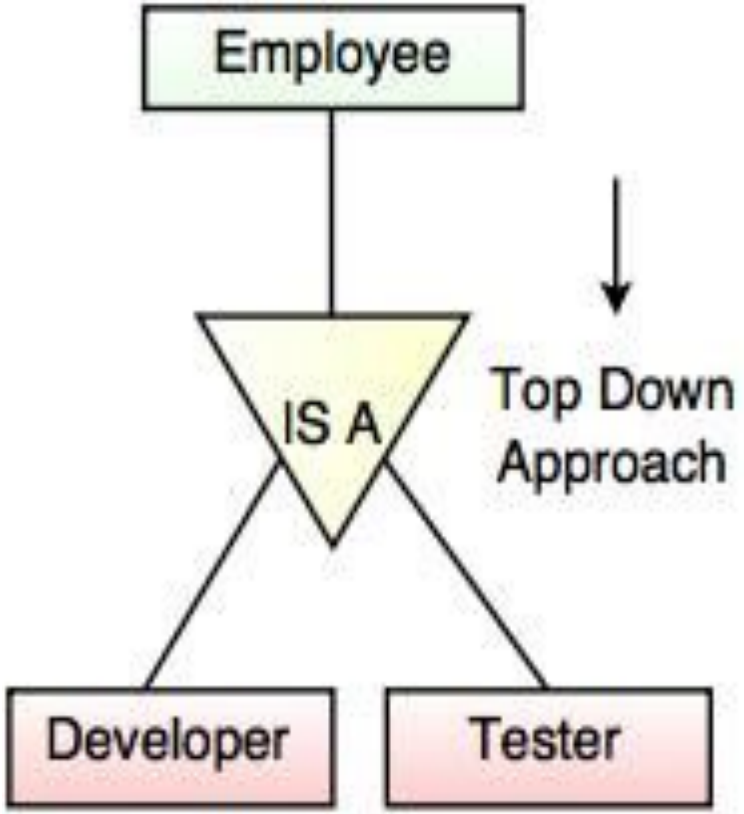
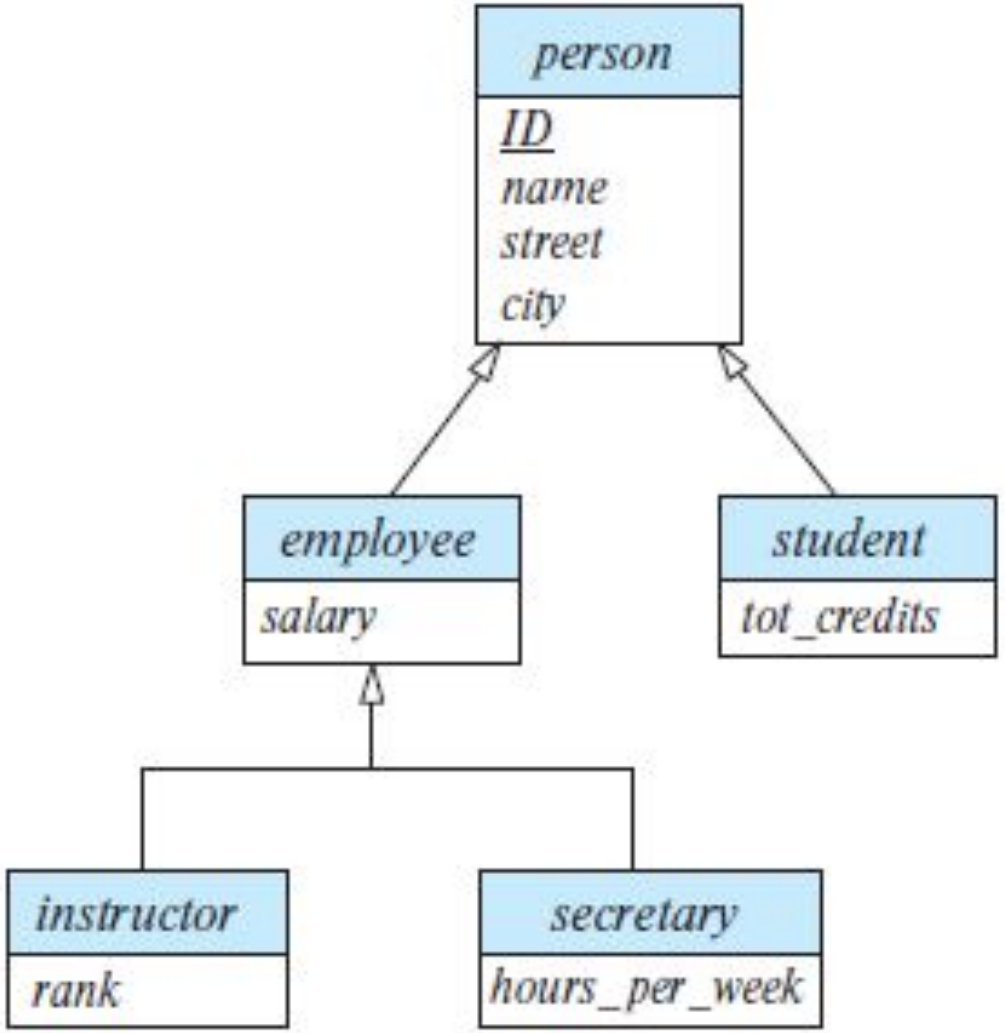


Fig. Specialization



Specialization and generalization.

Generalization

- Multiple entity sets are synthesized into a higher-level entity set, based on common features (bottom-up).
- Generalization is the process of generalizing the entities which contain the properties of all the generalized entities.
- It is a **bottom approach**, in which two lower level entities combine to form a higher level entity.
- Generalization is the reverse process of Specialization.
- It defines a general entity type from a set of specialized entity type.
- It minimizes the difference between the entities by identifying the common features.
  For example:

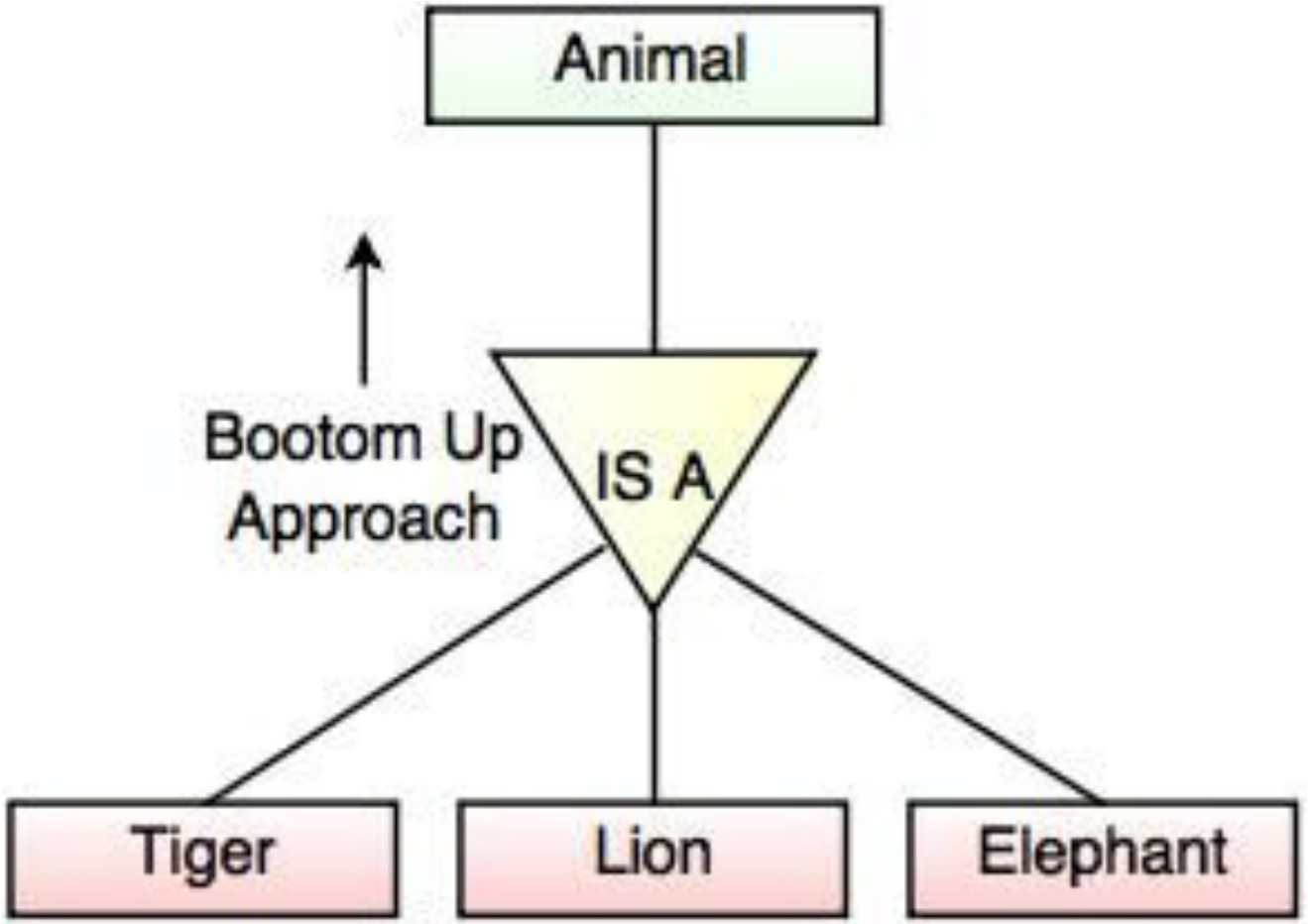


Fig. Generalization

In the example, Tiger, Lion, Elephant can all be generalized as Animals.

Aggregation

- Aggregation refers to the process by which entities are combined to form a single meaningful entity.
- The specific entities are combined because they do not make sense on their own.
- To establish a single entity, aggregation creates a relationship that combines these entities.
- Aggregation is a process that represent a relationship between a whole object and its component parts.
- It abstracts a relationship between objects and viewing the relationship as an object.
- It is a process when two entity is treated as a single entity.

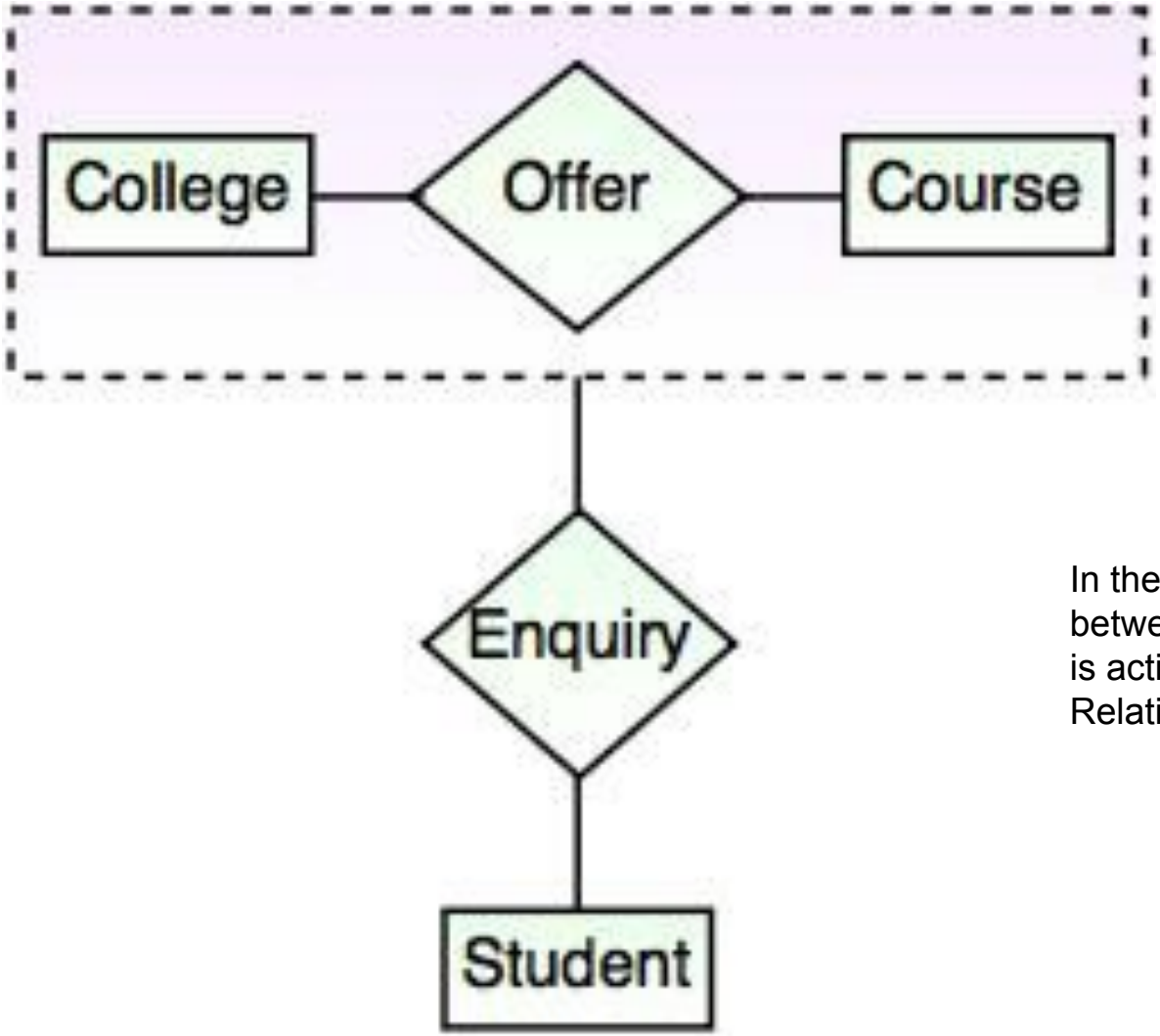


Fig. Aggregation

In the example, the relation between College and Course is acting as an Entity in Relation with Student.

Mapping specialization/generalization to relational tables

- Specialization/generalization relationship can be mapped to relational tables in three methods.
- To demonstrate the methods, we will take the student, postgraduate and undergraduate relationship.
- A student in the university has a registration number and a name.
- Only postgraduate students have supervisors. Undergraduates accumulates points through their coursework.

Features of EER Model

- EER creates a design more accurate to database schemas.
- It reflects the data properties and constraints more precisely.
- It includes all modeling concepts of the ER model.
- Diagrammatic technique helps for displaying the EER schema.
- It includes the concept of specialization and generalization.
- It is used to represent a collection of objects that is union of objects of different of different entity types.

Thank You

BACSE202 - Database Systems

Functional dependencies: Inference rules, Equivalence, and Minimal Cover - Axioms on Functional Dependencies

Dr. Somasundaram R

Assistant Professor Sr. Grade 2

Emp Id: 23770

Department of IoT

School of Computer Science and Engineering (SCOPE)

Functional dependency

Functional Dependencies

- It is a constraint between the 2 sets of attributes from the database
- It is a property of the semantic of the database
- DB designers specify the semantics by functional dependencies
- Functional dependency defines relationship among attributes of a relation schema.
- Example:
  - FD: x -> y ; x, y are attributes
  - x determines y or y is dependent on x

Functional Dependency

Formal Definition: **Attribute B is functionally dependant upon attribute A *(or a collection of attributes)* if a value of A determines a single value of attribute B at any one time.**

Formal Notation: **A → B** This should be read as **'A determines B'** or **'B is functionally dependant on A'**. A is called the *determinant* and B is called the *object of the determinant*.

Example:

staffNo	job	dept	dname
SL10	Salesman	10	Sales
SA51	Manager	20	Accounts
DS40	Clerk	20	Accounts
OS45	Clerk	30	Operations

Functional Dependencies

staffNo → job

staffNo → dept

staffNo → dname

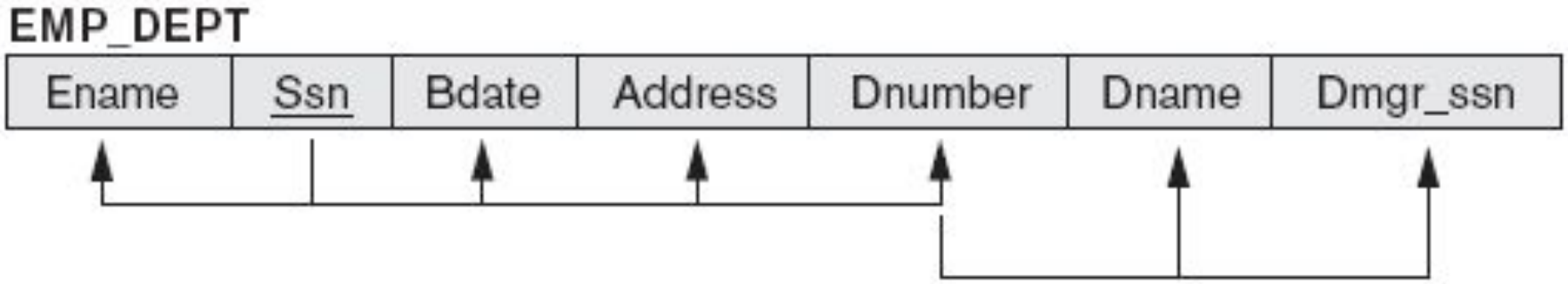
dept → dname

EMP_DEPT

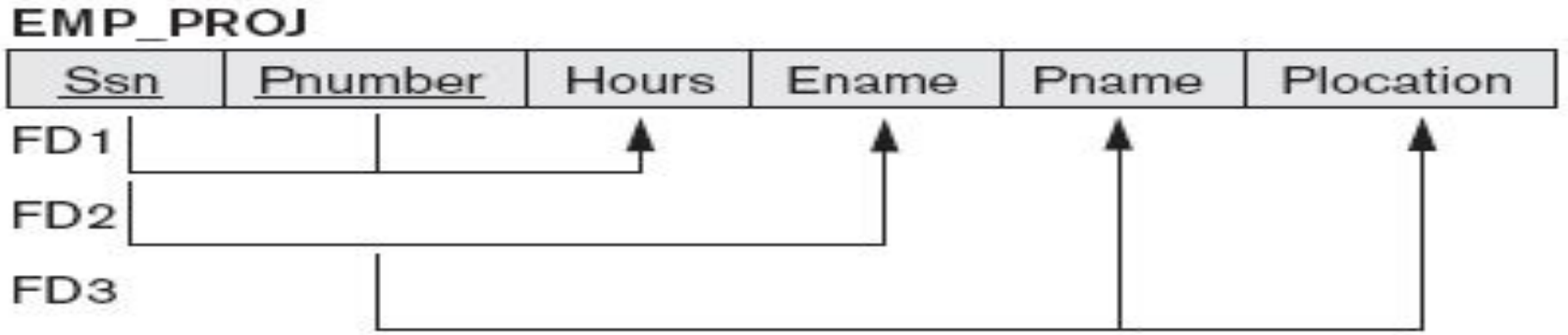
Ename	Ssn	Bdate	Address	Dnumber	Dname	Dmgr_ssn
Smith, John B.	123456789	1965-01-09	731 Fondren, Houston, TX	5	Research	333445555
Wong, Franklin T.	333445555	1955-12-08	638 Voss, Houston, TX	5	Research	333445555
Zelaya, Alicia J.	999887777	1968-07-19	3321 Castle, Spring, TX	4	Administration	987654321
Wallace, Jennifer S.	987654321	1941-06-20	291 Berry, Bellaire, TX	4	Administration	987654321
Narayan, Ramesh K.	666884444	1962-09-15	975 FireOak, Humble, TX	5	Research	333445555
English, Joyce A.	453453453	1972-07-31	5631 Rice, Houston, TX	5	Research	333445555
Jabbar, Ahmad V.	987987987	1969-03-29	980 Dallas, Houston, TX	4	Administration	987654321
Borg, James E.	888665555	1937-11-10	450 Stone, Houston, TX	1	Headquarters	888665555

EMP_PROJ

Ssn	Pnumber	Hours	Ename	Pname	Plocation
123456789	1	32.5	Smith, John B.	ProductX	Bellaire
123456789	2	7.5	Smith, John B.	ProductY	Sugarland
666884444	3	40.0	Narayan, Ramesh K.	ProductZ	Houston
453453453	1	20.0	English, Joyce A.	ProductX	Bellaire
453453453	2	20.0	English, Joyce A.	ProductY	Sugarland
333445555	2	10.0	Wong, Franklin T.	ProductY	Sugarland
333445555	3	10.0	Wong, Franklin T.	ProductZ	Houston
333445555	10	10.0	Wong, Franklin T.	Computerization	Stafford
333445555	20	10.0	Wong, Franklin T.	Reorganization	Houston
999887777	30	30.0	Zelaya, Alicia J.	Newbenefits	Stafford
999887777	10	10.0	Zelaya, Alicia J.	Computerization	Stafford
987987987	10	35.0	Jabbar, Ahmad V.	Computerization	Stafford
987987987	30	5.0	Jabbar, Ahmad V.	Newbenefits	Stafford
987654321	30	20.0	Wallace, Jennifer S.	Newbenefits	Stafford
987654321	20	15.0	Wallace, Jennifer S.	Reorganization	Houston
888665555	20	Null	Borg, James E.	Reorganization	Houston



{Ssn} ☐ {Ename, Bdate, Address, Dnumber}
{Dnumber} ☐ {Dname, Dmgr_ssn}



FD1: {Ssn, Pnumber} ☐ {Hours}
FD2: {Ssn} ☐ {Ename}
FD3: {Pnumber} ☐ {Pname, Plocation}

Armstrong’s rules (axioms)

Assume that R is a relation schema and X, Y and Z are subsets of R.

Reflexivity rule: If X is a super set of Y then X □ Y.

Augmentation rule: If X □ Y then XZ □ YZ.

Transitivity rule: If X □ Y & Y □ Z then X □ Z.

Additional rules

Union rule: If X → Y and X → Z then X → YZ

Decomposition rule: If X → YZ then X → Y and X → Z

Pseudo transitivity rule:

If X → Y and YZ → V then XZ → V

Composition rule: If X → Y and V → W then XV → YW

Closure of a set of attributes

- Assume that X is a subset of a schema R & F is a set of functional dependencies that hold on R.
- Closure of X is denoted by X^+.
- Closure of X (X^+) is the maximal (biggest) super set of X such that X □ X^+.
- Closure is a set of attributes that are functionally determines by the attribute set

Algorithm for computing X^+

Given R & F find X^+

Algorithm:

Begin

$X^+ := X$

Repeat

For each functional dependency Y □ Z ε F,

if X^+ contains Y then

$X^+ := X^+ \cup Z$

Until X^+ does not change.

End.

Example:

Let R={A,B,C,D,E}

F = { A -> B , A -> C; B -> D; C -> D; D -> E }

Find A^+

Step-1: A^+ = { A }

Step-2: A^+ = { A , B , C}

Step-3: A^+ = { A , B, C, D}

Step-4: A^+ = { A , B, C, D}

Step-5: A^+ = { A , B, C, D, E}

Similarly find B^+, C^+, D^+

Example

R(A, B, C, G, H, I)

F = {A □ B, A □ C, CG □ H, CG □ I, B □ H},

Find out (i) A^+ & (ii) $(AG)^+$

Closure

R(A, B, C, G, H, I)

F = {A □ B, A □ C, CG □ H, CG □ I, B □ H},

$A^+ = A$

Since A □ B so $A^+ = AB$

Since A □ C so $A^+ = ABC$

Consider CG □ H, CG is not a subset of A^+ so A^+ remains unaltered

Consider CG □ I, CG is not a subset of A^+ so A^+ remains unaltered

Consider B □ H, B is a subset of A^+ so $A^+ = ABCH$

Closure

R(A, B, C, G, H, I)

F = {A □ B, A □ C, CG □ H, CG □ I, B □ H},

$(AG)^+$ = AG

= AGB (because of A □ B)

= AGBC (because of A □ C)

= AGBCH (because of CG □ H)

= AGBCHI (because of CG □ I)

which remains unaltered despite B □ H

$(AG)^+$ = R

Super Key & Key

- If the closure of the attributes/set of attributes determines all the attributes of the relation then it is the super key
- Since $(AG)^+ = R$ hence AG is a super key of R
- To find out whether it is a key, we have seen that $A^+ \neq R$ & also note that $G^+ \neq R$
- So AG is a key of R
- If X is a subset of a relation schema R then X is a super key of R if $X^+ = R$

Key

Input: A relation schema R and a set of functional dependencies F on the attributes of R.

1. Set K := R;

2. For each attribute A in K

Begin

Compute $(K - A)^+$ with respect to F;

If $(K - A)^+$ contains all the attributes in R,

then set K := K – {A};

End.

Example

R(A, B, C, D, E) & F = {A □ C, C □ D, B □ E}

Find out a key of R.

Answer: Key = AB

Key

$$\mathcal{F} = \{A \rightarrow C, C \rightarrow D, B \rightarrow E\}$$

Initialize K := ABCDE.

Suppose that we remove A from ABCDE and compute $(BCDE)^+$.

We find that $(BCDE)^+ = (BCDE) \neq R$ and hence A cannot be removed from ABCDE.

Suppose that we remove B from ABCDE and compute $(ACDE)^+$.

We find that $(ACDE)^+ = ACDE \neq R$ and hence B too cannot be removed from ABCDE.

Suppose that we remove C from ABCDE and compute $(ABDE)^+$.

We find that $(ABDE)^+$= ABDEC = R and hence C can be removed from ABCDE. So K is now reduced to ABDE.

Key

Suppose that we remove D from ABDE and compute $(ABE)^+$.

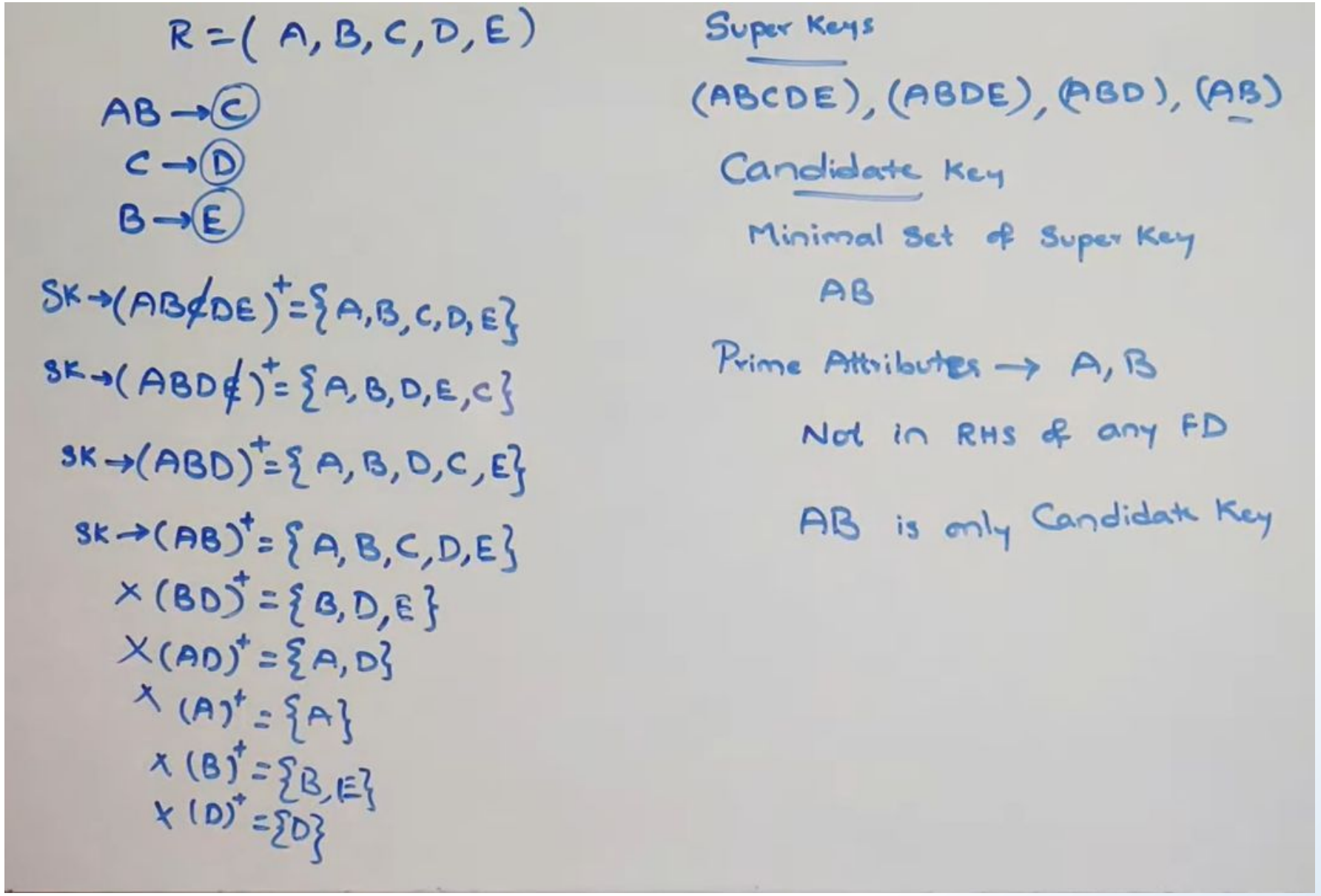
We find that $(ABE)^+$ = ABECD = R and hence D can be removed from ABDE. So K is reduced to ABE.

Suppose that we remove E from ABE and compute $(AB)^+$.

We find that $(AB)^+$ = ABCDE = R and hence E can be removed from ABE.

So a key of the relation schema R is AB.

Find the Superkeys and Candidate Keys



Example

R(A, B, C, D, E, H) & F = {A □ B, BC □ D, E□ C, D□A }

Find out the candidate key of R.

$R = (A, B, C, D, E, H)$

$A \to B$ ✓

$BC \to D$ ✓

$E \to C$

$D \to A$ ✓

SK $(ABCDEH)^+ = \{A, B, C, D, E, H\}$

SK $(ACDEH)^+ = \{A, C, D, E, H, B\}$

SK $(ADEH)^+ = \{A, D, E, H, B, C\}$

SK $(DEH)^+ = \{D, E, H, A, B, C\}$

$\times (DE)^+ = \{D, E, A, C, B\}$

$\times (EH)^+ = \{E, H, C\}$

$\times (DH)^+ = \{D, H, A, B\}$

$\times (D)^+ = \{D, A, B\}$

$\times (E)^+ = \{E, C\}$

$\times (H)^+ = \{H\}$

SK → ABCDEH, ACDEH, ADEH, DEH, BCEH, BEH, AEH

Candidate Key → DEH, BEH, AEH

Prime Attributes → D, E, H

$BC \to D$ (PA as RHS)

Replace PA with LHS

DEH

$(BCEH)^+ = \{B, C, E, H, D, A\}$ SK

$(BEH)^+ = \{B, E, H, C, D, A\}$ SK

$\times (BE)^+ = \{B, E, C, D, A\}$

$\times (EH)^+ = \{E, H, C\}$

$\times (BH)^+ = \{B, H\}$

$\times (B)^+ = \{B\}$

$\times (E)^+ = \{E, C\}$

$\times (H)^+ = \{H\}$

Prime Attributes → B, E, H

$A \to B$ (PA as RHS)

Replace PA with LHS

BEH ⟹ AEH

$(AEH)^+ = \{A, E, H, B, C, D\}$ SK

$(AE)^+ = \{A, E, B, C, D\}$ ×

$(EH)^+ = \{E, H, C\}$ ×

$(AH)^+ = \{A, H, B\}$ ×

$(A)^+ = \{A, B\}$ ×

$(E)^+ = \{E, C\}$ ×

$(H)^+ = \{H\}$ ×

Prime Attribute – A, E, H

$D \to A$ (PA as RHS)

Replace PA with LHS

AEH ⟹ DEH

Example

R(A, B, C, D) & F = {A □ B, B □ C, C □ BD}.

Find out candidate key(s) of R.

Key

$ABCD^+ = R$

$ACD^+ = R$

$AD^+ = R$

$A^+ = ABCD = R$ and so A is a key.

$B^+ = BCD \neq R$ and so B cannot be a key.

$C^+ = CBD \neq R$ and so C cannot be a key.

The super keys are : ABCD, ACD, AD, A

The minimal super key is the candidate key, so R has one minimal super key and it is A

Example

R(A, B, C, D, E, G, H) &

F = {A □ B, B □ ACE, C □ BGD, G □ DH}.

Find out Candidate key(s) of R.

A^+=AB

ABCE

ABCEGD

ABCEGDH=R

Key

A^+ = ABCEGDH = R and so A is a key.

B^+ = BACEGDH = R & so B is also a key.

C^+ = CBGDHAE = R and so C also is a key.

G^+ = GDH and so G is not a key.

So there are three keys of R and they are A, B and C.

Example

R(A, B, C, D, E) &

F = {A □ B, B □ A, B □ C, D □ A}.

Find out key(s) of R.

Key

A^+= ABC ≠ R so A cannot be a key.

B^+ = BAC ≠ R, so B cannot be a key.

D^+ = DABC ≠ R and so D cannot be a key.

$(DE)^+$ = DEABC = R and so DE is a key of R.

Inference

- R & F
- X → Y not a member of F
- Is it possible to infer X → Y from F?
- Find out X^+ with respect to F
- If X^+ contains Y then X → Y can be inferred from F.

Example

- F = {A □ C, AC □ D, E □ AD, E □ H}
- A □ CD – Can it be inferred from F?
- A^+ = ACD
- A □ CD can be inferred F
- E □ AH can be inferred F, because E^+ = EADCH

Equivalence

- R, F_1 & F_2
- $F_1 \equiv F_2$
- Two sets of functional dependency are said to be equivalent if every functional dependency of one set can be derived from the other set of functional dependency.

Example

R(A, B, C, D),

F_1 = {A □ B, B □ C, AC □ D},

F_2 = {A □ B, B □ C, A □ D}.

Are F_1 & F_2 equivalent?

Solution

- We find that the functional dependency A □ D of F_2 can be inferred from F_1 because
- A^+ with respect to F_1 is
- A^+ := A = AB = ABC = ABCD which contains D and hence A □ D can be inferred from F_1.
- Again AC □ D of F_1 can be inferred from F_2 because
- $(AC)^+$ with respect to F_2 is
- $(AC)^+$:= AC = ABC = ABCD which contains D and hence AC □ D of F_1 can be inferred from F_2.
- Hence F_1 and F_2 are equivalent.

Example

- R(A, B, C)
- F_1 = {A □ B, A □ C} & F_2 = {A □ B, B □ C}
- Are F_1 & F_2 equivalent?

Solution

- The first functional dependency in the two sets are the same
- Consider A □ C of F_1
- Compute A^+ w.r.t. F_2
- A^+ = ABC
- hence A □ C can be derived from F_2

Solution….

- Consider B □ C of F_2
- Compute B^+ w.r.t. F_1
- B^+ = B which does not contain C
- Hence B □ C cannot be inferred from F_1
- So F_1 and F_2 are not equivalent

Thank You

BACSE202 • DATABASE SYSTEMS

Functional Dependencies

Inference • Equivalence • Minimal Cover

Dr. Somasundaram R

Learning Outcomes

By the end of this class, students will be able to:

1. Explain how new Functional Dependencies are inferred from existing ones
2. Apply attribute closure to test whether an FD can be inferred
3. Determine whether two sets of Functional Dependencies are equivalent
4. Explain the purpose and rules of a Minimal Cover
5. Compute a Minimal Cover for a given set of Functional Dependencies

Inference

Definition: Inference is the process of deriving a new Functional Dependency (FD) from existing Functional Dependencies, using Armstrong's Rules or attribute closure.

SIMPLE MEANING: *Inference means discovering a new FD from the given FDs.*

KEY POINT

If Y is present in X^+, then X → Y can be inferred.

Inference by Transitivity

Given:

$A \rightarrow B$

$B \rightarrow C$

Question: Can we infer $A \rightarrow C$?

SOLUTION

$A \rightarrow B$ and $B \rightarrow C$

Using Transitivity:

$A \rightarrow C$

RESULT

$A \rightarrow C$

is inferred.

Inference by Attribute Closure

Given: $F = \{ A \rightarrow B,\ B \rightarrow C,\ C \rightarrow D \}$

Question: *Can $A \rightarrow D$ be inferred?*

FIND A^+ (Closure of A)

Start	$A^+ = \{ A \}$
Apply $A \rightarrow B$	$A^+ = \{ A, B \}$
Apply $B \rightarrow C$	$A^+ = \{ A, B, C \}$
Apply $C \rightarrow D$	$A^+ = \{ A, B, C, D \}$

CONCLUSION

$D \in A^+$

So $A \rightarrow D$ can be inferred.

Equivalence

Definition: Two sets of Functional Dependencies are equivalent if every FD in one set can be derived from the other set, and vice versa.

SIMPLE MEANING: *Both FD sets represent the same information.*

VISUALIZING EQUIVALENCE



Equivalence — Equivalent Case

F1

$A \rightarrow B$

$B \rightarrow C$

F2

$A \rightarrow B$

$B \rightarrow C$

$A \rightarrow C$

$F1 \equiv F2$

Equivalent

CHECK

From F1:

$A \rightarrow B$ and $B \rightarrow C$

$\Rightarrow$ $A \rightarrow C$ can be derived

From F2:

$A \rightarrow B$ $B \rightarrow C$ and $A \rightarrow C$

All required dependencies are already available

Equivalence — Not Equivalent Case

F1

$A \rightarrow B$

$A \rightarrow C$

F2

$A \rightarrow B$

$B \rightarrow C$

$F1 \not\equiv F2$

Not Equivalent

CHECK — Can $B \rightarrow C$ be derived from F1?

Compute: $B^+ = \{ B \}$

B^+ cannot reach C, so

B → C cannot be derived from F1.

Minimal Cover

Definition: A Minimal Cover is the smallest set of Functional Dependencies that is equivalent to the original set.

SIMPLE MEANING: *Remove all unnecessary attributes and duplicate dependencies without changing the meaning.*

RULES FOR FINDING A MINIMAL COVER

1. Split the right-hand side into single attributes
2. Remove unnecessary attributes from the left-hand side
3. Remove redundant Functional Dependencies

Minimal Cover — Splitting the RHS

Given: F = { A → BC, B → C }

STEP 1 — SPLIT THE RIGHT-HAND SIDE

A → B

A → C

B → C

Now check whether any dependency is redundant. None are redundant here.

MINIMAL COVER

A → B

A → C

B → C

Minimal Cover — Removing a Redundant FD

Given: F = { A → B, A → C, A → BC }

A → B	A → C	A → BC

A → BC is unnecessary, because A → B and A → C already give the same information.

MINIMAL COVER

A → B	A → C

Comparison Table

Topic	Definition	Purpose
Inference	Deriving a new Functional Dependency from existing FDs	To discover new dependencies
Equivalence	Two FD sets represent the same information	To compare two FD sets
Minimal Cover	Smallest equivalent set of Functional Dependencies	To remove redundancy before normalization

Easy Memory Trick

Inference	→ **Find a new FD**
Equivalence	→ **Compare two FD sets**
Minimal Cover	→ **Reduce the FD set to the minimum without changing its meaning**

SUMMARY

Today We Learned

- Inference of Functional Dependencies
- Attribute Closure (X^+)
- Equivalence of FD Sets
- Testing FD Set Equivalence
- Minimal Cover
- Rules for Computing a Minimal Cover

BACSE202 - Database Systems

Normalization: First, Second and Third Normal Forms - Boyce Codd Normal Form, Multi-Valued Dependency and Fourth Normal Form - Join Dependency and Fifth Normal Form

Dr. Somasundaram R

Assistant Professor Sr. Grade 2

Emp Id: 23770

Department of IoT

School of Computer Science and Engineering (SCOPE)

Purpose of Normalisation

- To avoid redundancy by storing each **'fact'** within the database only once.
- To put data into a form that conforms to relational principles (e.g., single valued attributes, each relation represents one entity) - **no repeating groups**.
- To put the data into a form that is more able to accurately accommodate change.
- To avoid certain updating **'anomalies'**.
- To facilitate the enforcement of data constraints.

Redundancy and Data Anomalies

Redundant data is where we have stored the same 'information' more than once. i.e., the redundant data could be removed without the loss of information.

Example: We have the following relation that contains staff and department details:

staffNo	job	dept	dname	city
SL10	Salesman	10	Sales	Stratford
SA51	Manager	20	Accounts	Barking
DS40	Clerk	20	Accounts	Barking
OS45	Clerk	30	Operations	Barking



Such 'redundancy' could lead to the following 'anomalies'

Insert Anomaly: We can't insert a dept without inserting a member of staff who works in that department

Update Anomaly: We could change the name of the dept that SA51 works in without simultaneously changing the dept that DS40 works in.

Deletion Anomaly: By removing employee SL10 we have removed all information pertaining to the Sales dept.

Repeating Groups

A repeating group is an attribute (or set of attributes) that can have more than one value for a primary key value.

Example: We have the following relation that contains staff and department details and a list of telephone contact numbers for each member of staff.

staffNo	job	dept	dname	city	contact number
SL10	Salesman	10	Sales	Stratford	**018111777, 018111888, 079311122**
SA51	Manager	20	Accounts	Barking	**017111777**
DS40	Clerk	20	Accounts	Barking	
OS45	Clerk	30	Operations	Barking	**079311555**

Repeating Groups are not allowed in a relational design, since all attributes have to be 'atomic' - i.e., there can be only one value per cell in a table!

Normalisation - Relational Model

In order to comply with the relational model principles it is necessary to

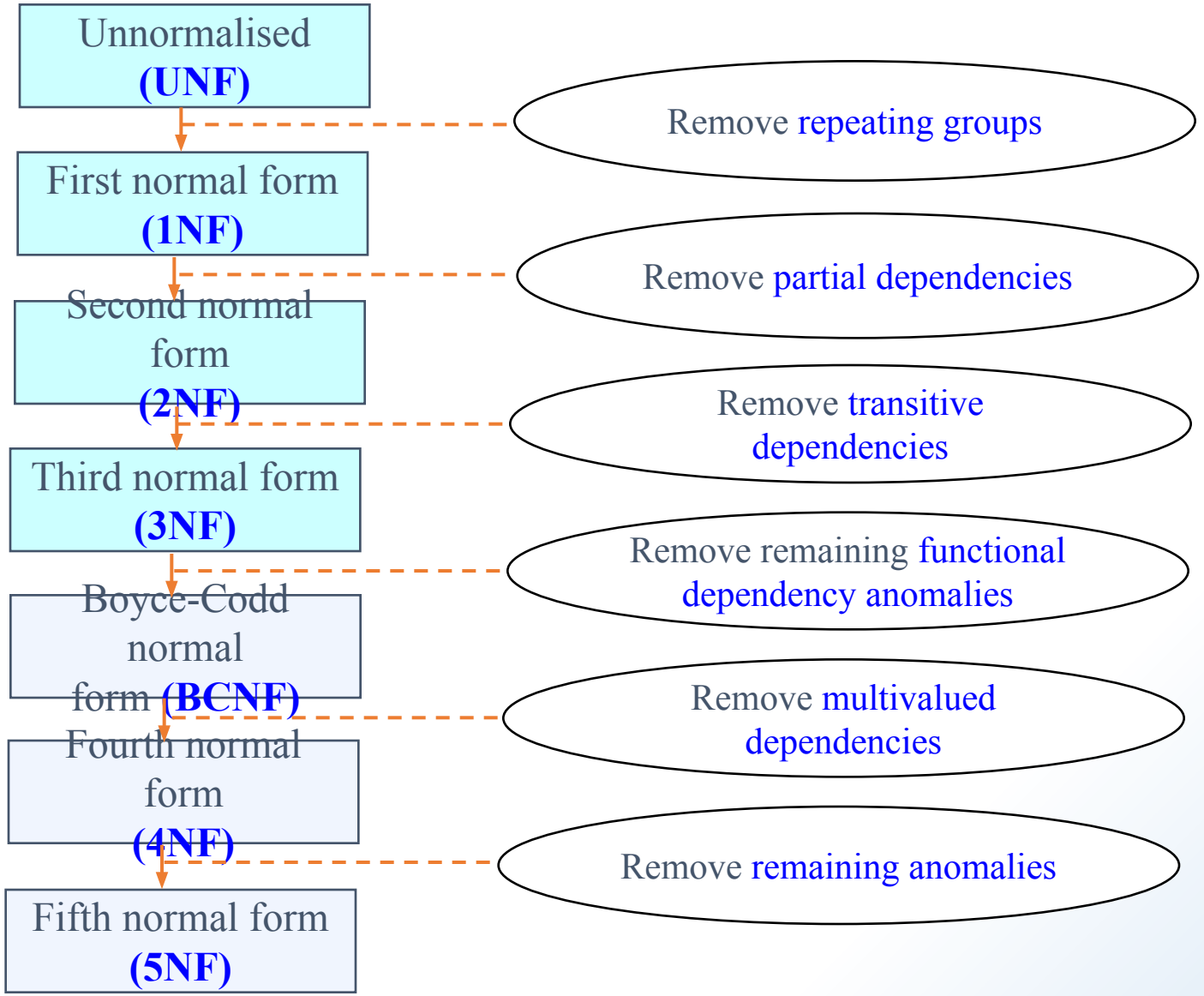
1) remove repeating groups and
2) avoid redundancy and data anomalies by removing full, partial and transitive functional dependencies.

Relational Database Design: All attributes in a table must be atomic, and solely dependant upon the fully primary key of that table.

NORMALISATION ACHIEVES THIS!

THE KEY, THE WHOLE KEY, AND NOTHING BUT THE KEY!

Stages of Normalisation



Unnormalised Normal Form (UNF)

Definition: A relation is unnormalised when it has not had any normalisation rules applied to it, and it suffers from various anomalies.

This only tends to occur where the relation has been designed using a 'bottom-up approach'. *i.e., the capturing of attributes to a 'Universal Relation' from a screen layout, manual report, manual document, etc...*

Unnormalised Normal Form (UNF)

ORDER

Customer No: 001964
Name: Mark Campbell
Address: 1 The House
Leytonstone
E11 9ZZ

Order Number: 00012345
Order Date: 14-Feb-2002

Product Number	Product Description	Unit Price	Order Quantity	Line Total
T5060	Hook	5.00	5	25.00
PT42	Bolt	2.50	10	20.50
QZE48	Spanner	20.00	1	20.00

Order Total: 65.50

ORDER (order-no, order-date, cust-no, cust-name, cust-add,
*(prod-no, prod-desc, unit-price, ord-qty, line-total)**, order-total

Rules for the relation R in First Normal Form

- Each attribute should contain atomic values
- A column (attribute) should contain value from the same domain
- Attribute/Column names should be unique within a relation
- No ordering of rows and columns (Row/Column order has no meaning)
- No duplicate rows

First Normal Form (1NF)

Definition: A relation is in 1NF if, and only if, all its underlying attributes contain atomic values only.

Remove repeating groups into a new relation

A repeating group is shown by a pair of brackets within the relational schema.

ORDER (order-no, order-date, cust-no, cust-name, cust-add,
(prod-no, prod-desc, unit-price, ord-qty, line-total),* order-total

Steps from UNF to 1NF:

- Remove the outermost repeating group (and any nested repeated groups it may contain) and create a new relation to contain it.
- Add to this relation a copy of the PK of the relation immediately enclosing it.
- Name the new entity *(appending the number 1 to indicate 1NF)*
- Determine the PK of the new entity
- Repeat steps until no more repeating groups.

Example - UNF to 1NF

ORDER (order-no, order-date, cust-no, cust-name, cust-add,
(prod-no, prod-desc, unit-price, ord-qty, line-total),* order-total

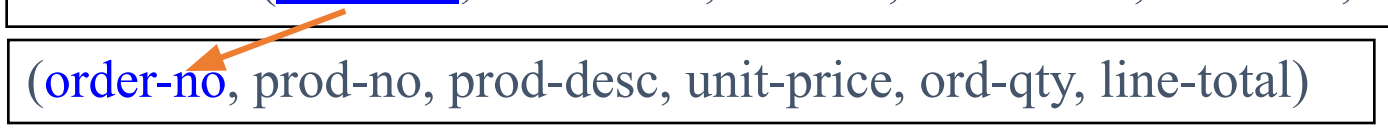
1. Remove the outermost repeating group (and any nested repeated groups it may contain) and create a new relation to contain it. *(rename original to indicate 1NF)*

ORDER-1 (order-no, order-date, cust-no, cust-name, cust-add, order-total

(prod-no, prod-desc, unit-price, ord-qty, line-total)

2. Add to this relation a copy of the PK of the relation immediately enclosing it.

ORDER-1 (order-no, order-date, cust-no, cust-name, cust-add, order-total



3. Name the new entity *(appending the number 1 to indicate 1NF)*

ORDER-LINE-1 (order-no, prod-no, prod-desc, unit-price, ord-qty, line-total)

4. Determine the PK of the new entity

ORDER-LINE-1 (order-no, prod-no, prod-desc, unit-price, ord-qty, line-total)

Second Normal Form (2NF)

Definition: A relation is in 2NF if, and only if, it is in 1NF and every non-primary-key attribute is fully functionally dependent on the primary key, then the relation is in Second Normal Form (2NF).

Remove partial functional dependencies into a new relation

Steps from 1NF to 2NF:

- Remove the offending attributes that are only partially functionally dependent on the composite key, and place them in a new relation.
- Add to this relation a copy of the attribute(s) which are the determinants of these offending attributes. These will automatically become the primary key of this new relation.
- Name the new entity *(appending the number 2 to indicate 2NF)*
- Rename the original entity *(ending with a 2 to indicate 2NF)*

Rules for the relation R in Second Normal Form

- **Should be in First Normal form**
- **No partial dependency should exist**
- **Meaning of partial dependency :**
  - If Proper subset of any candidate key -> (determines) non prime attribute then there exist a partial dependency.

Example - 1NF to 2NF

ORDER-LINE-1 (order-no, prod-no, prod-desc, unit-price, ord-qty, line-total)

1. Remove the offending attributes that are only partially functionally dependent on the composite key, and place them in a new relation.

ORDER-LINE-1 (order-no, prod-no, ord-qty, line-total)

(prod-desc, unit-price)

2. Add to this relation a copy of the attribute(s) which determines these offending attributes. These will automatically become the primary key of this new relation..

ORDER-LINE-1 (order-no, prod-no, ord-qty, line-total)



3. Name the new entity *(appending the number 2 to indicate 2NF)*

PRODUCT-2 (prod-no, prod-desc, unit-price)

4. Rename the original entity *(ending with a 2 to indicate 2NF)*

ORDER-LINE-2 (order-no, prod-no, ord-qty, line-total)

Third Normal Form (3NF)

Definition: A relation is in 3NF if, and only if, it is in 2NF and every non-primary-key attribute is non transitively dependent on the primary key, then the relation is in Third Normal Form (3NF)

Remove transitive dependencies into a new relation

Steps from 2NF to 3NF:

- Remove the offending attributes that are transitively dependent on non-key attribute(s), and place them in a new relation.
- Add to this relation a copy of the attribute(s) which are the determinants of these offending attributes. These will automatically become the primary key of this new relation.
- Name the new entity *(appending the number 3 to indicate 3NF)*
- Rename the original entity *(ending with a 3 to indicate 3NF)*

Example - 2NF to 3NF

ORDER-2 (order-no, order-date, cust-no, cust-name, cust-add, order-total

1. Remove the offending attributes that are transitively dependent on non-key attributes, and place them in a new relation.

ORDER-2 (order-no, order-date, cust-no, order-total

(cust-name, cust-add)

2. Add to this relation a copy of the attribute(s) which determines these offending attributes. These will automatically become the primary key of this new relation..

ORDER-2 (order-no, order-date, cust-no, order-total



(cust-no, cust-name, cust-add)

3. Name the new entity *(appending the number 3 to indicate 3NF)*

CUSTOMER-3 (cust-no, cust-name, cust-add)

4. Rename the original entity *(ending with a 3 to indicate 3NF)*

ORDER-3 (order-no, order-date, cust-no, order-total

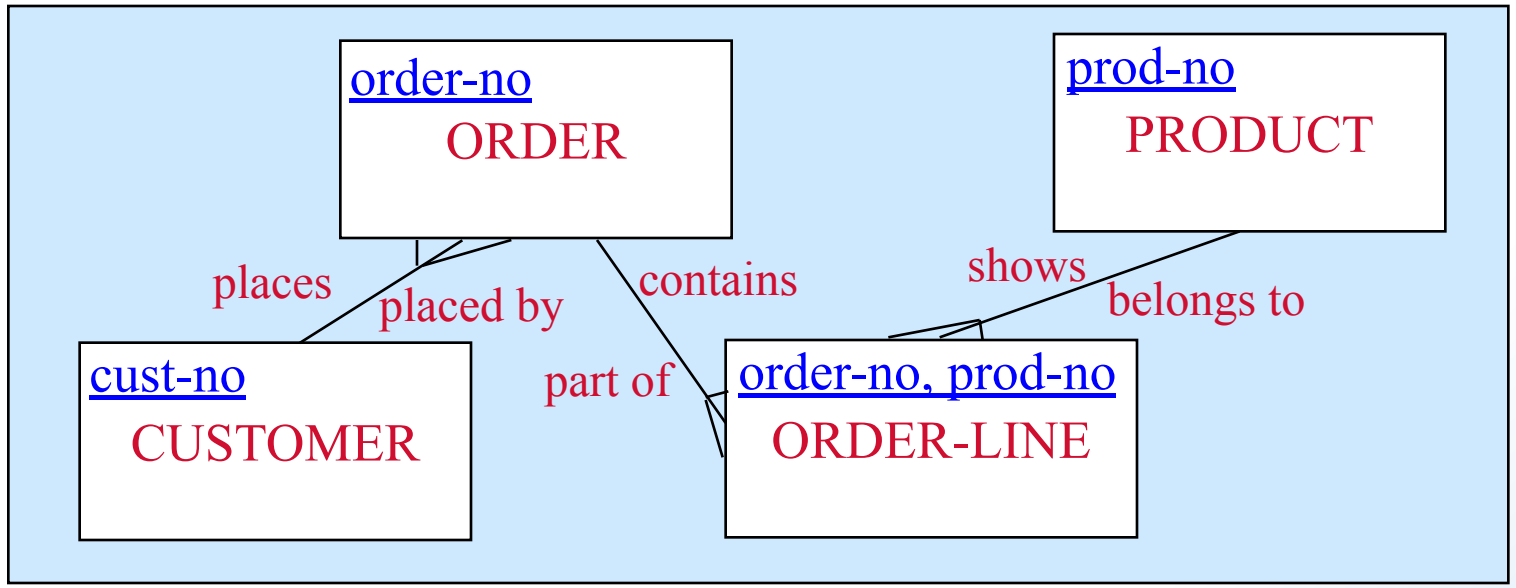
Example - Relations in 3NF

ORDER-3 (order-no, order-date, cust-no, order-total

CUSTOMER-3 (cust-no, cust-name, cust-add)

PRODUCT-2 (prod-no, prod-desc, unit-price)

ORDER-LINE-2 (order-no, prod-no, ord-qty, line-total)



ANOTHER DEFINITION

- A Relation R is said to be in 3NF if and only if it is 2NF and for each of its non-trivial functional dependencies at least one of the following condition holds on the FD

  i) L.H.S is the Super Key

  ii) R.H.S is Prime Attribute

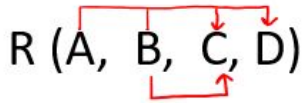
Note: Trivial means A->B then B ⊆ A { contained in}

non-Trivial means B not contained in A

Problems in normalization

- Given a relation R(A, B, C, D) and Functional Dependency set FD = { AB $\rightarrow$ CD, B $\rightarrow$ C }, determine whether the given R is in 2NF? If not convert it into 2 NF.

- **Solution: Let us construct an arrow diagram on R using FD to calculate the candidate key.**



- **From above arrow diagram on R, we can see that an attributes AB is not determined by any of the given FD, hence AB will be the integral part of the Candidate key, i.e. no matter what will be the candidate key, and how many will be the candidate key, but all will have W compulsory attribute.**

- **Let us calculate the closure of AB**

- **$AB^+ = ABCD$**

- **Since the closure of AB contains all the attributes of R, hence AB is Candidate Key**

- **From the definition of Candidate Key(Candidate Key is a Super Key whose no proper subset is a Super key)**

- Since all key will have AB as an integral part, and we have proved that AB is Candidate Key, Therefore, any superset of AB will be Super Key but not Candidate key.

- Hence there will be only one candidate key AB

- Definition of 2NF: No non-prime attribute should be partially dependent on Candidate Key
- If Proper subset of candidate key -> non prime attribute then it is not in 2NF
- Since R has 4 attributes: - A, B, C, D, and Candidate Key is AB, Therefore, prime attributes (part of candidate key) are A and B while a non-prime attribute are C and D
- a) FD: AB $\longrightarrow$ CD satisfies the definition of 2NF, that non-prime attribute(C and D) are fully dependent on candidate key AB
- b) FD: B $\longrightarrow$ C does not satisfy the definition of 2NF, as a non-prime attribute(C) is partially dependent on candidate key AB(i.e. key should not be broken at any cost)
- As FD B $\longrightarrow$ C, the above table R(A, B, C, D) is not in 2NF
- Convert the table R(A, B, C, D) in 2NF:
- Since FD: B $\longrightarrow$ C, our table was not in 2NF, let's decompose the table
-

- R1(B, C)
- Since the key is AB, and from FD AB $\rightarrow$ CD, we can create R2(A, B, C, D) but this will again have a problem of partial dependency B $\rightarrow$ C, hence R2(A, B, D).
- Finally, the decomposed table which is in 2NF
- a) R1(B, C)
- b) R2(A, B, D)

Exercise problem 1

Consider the following relation schema:

CAR_SALE(Car_id, Date_sold, Salesperson_id, Commission_pct, Discount_amt).

Additional functional dependencies are

Date_sold → Discount_amt,

Salesperson_id → Commission_pct.

Based on the given primary key, is this relation in 1NF, 2NF, or 3NF? Why or why not? How would you successively normalize it completely?

CAR_SALE(Car_id, Date_sold, Salesperson_id, Commission_pct, Discount_amt)

Date_sold □ Discount_amt,

Salesperson_id □ Commission_pct.

R_1(Salesperson_id , Commission_pct.)

R_2 (Car_id, Date_sold, Salesperson_id, Discount_amt)

R_3(Date_sold, Discount_amt) R_4 (Car_id, Date_sold, Salesperson_id)

$D = \{R_1, R_3, R_4)$

Exercise problem 2

Consider the following relation for published books:

BOOK (Book_title, Author_name, Book_type, List_price, Author_affil, Publisher).

Author_affil refers to the affiliation of author.

Suppose that the following dependencies exist:

Book_title → Publisher, Book_type

Book_type → List_price

Author_name → Author_affil.

What normal form is the relation in? Explain your answer.

Apply normalization until you cannot decompose the relations further. State the reasons behind each decomposition.

BOOK (Book_title, Author_name, Book_type, List_price, Author_affil, Publisher)

Book_title → Publisher, Book_type
Book_type → List_price
Author_name → Author_affil.

R_1(Book_title, Publisher, Book_type)

R_2(Book_title, Author_name, List_price, Author_affil)

R_3(Author_name, Author_affil.)

R_4(Book_title, Author_name, List_price)

$D = \{R_1, R_3, R_4\}$

$D = \{R_1, R_3, R_4, R_5(\text{Book_type}, \text{List_price})\}$

Alternative decomposition

BOOK (Book_title, Author_name, Book_type, List_price, Author_affil, Publisher)

Book_title → Publisher, Book_type
Book_type → List_price
Author_name → Author_affil.

R_1(Book_type, List_price)

R_2(Book_title, Author_name, Book_type, Author_affil, Publisher)

R_3(Author_name, Author_affil.)

R_4(Book_title, Author_name, Book_type, Publisher)

$D = \{R_1, R_3, R_4\}$

Exercise problem 3

Consider the universal relation

R = {A, B, C, D, E}

and the set of functional dependencies

F = {AB□C, A□DE}.

What is the key for R? Decompose R into 2NF and then 3NF relations.

Solution

$(AB)^+ = ABCDE = R$

$A^+ = ADE \neq R$ $B^+ = B$

AB is a key of R

AB □ C, A □ DE

$R_1(\underline{A}, D, E)$ $R_2(\underline{A}, \underline{B}, C)$

$D = \{R_1, R_2\}$

Exercise problem 4

Consider the universal relation

R = {A, B, C, D, E, G, H, I, J, K}

and the set of functional dependencies

F = {AB□C, A□DE, B□K, K□GH, D □ IJ}.

What is the key for R? Decompose R into the highest possible normal form.

Solution

R(A, B, C, D, E, G, H, I, J, K)

Key of the schema is AB

{AB□C, A□DE, B□K, K□GH, D □ IJ}

R_1(D, I, J) R_2(A, B, C, D, E, G, H, K)

R_3(A, D, E) R_4(A, B, C, G, H, K)

R_5(K, G, H) R_6(A, B, C, K)

R_7(B, K) R_8(A, B, C)

D = {R_1, R_3, R_5, R_7, R_8}

Thank You

BACSE202 - Database Systems

Normalization: First, Second and Third Normal Forms - Boyce Codd Normal Form, Multi-Valued Dependency and Fourth Normal Form - Join Dependency and Fifth Normal Form

Dr. Somasundaram R

Assistant Professor Sr. Grade 2

Emp Id: 23770

Department of IoT

School of Computer Science and Engineering (SCOPE)

BCNF

- A Relation R is said to be in in Boyce-Codd normal form (BCNF) if and only if it is in 3NF and every determinant is a candidate key.

Anomalies can occur in relations in 3NF if there is a composite key in which part of that key has a determinant which is not itself a candidate key.

This can be expressed as R(A,B,C), C--->A where:
The relation R contains attributes A, B and C.
A and B form a candidate key.
C is the determinant for A (A is functionally dependent on C).
C is not part of any key.

Anomalies can also occur where a relation contains several candidate keys where:

- The keys contain more than one attribute (they are composite keys).
- An attribute is common to more than one key.

Example to understand BCNF:-

Consider the following Relation:

campus	course	class	time	room/bldg
East	English 101	1	8:00-9:00	212 PRP
East	English 101	2	10:00-11:00	305 PRP
West	English 101	3	8:00-9:00	102 SJT

Note that no two buildings on any of the university campuses have the same name, thus ROOM/BLDG-->CAMPUS. As the determinant is not a candidate key this Relation is NOT in Boyce-Codd normal form.

This relation should be decomposed into the

R1(course, class, room/bldg, time)

course	class	time	room/bldg
English 101	1	8:00-9:00	212 PRP
English 101	2	10:00-11:00	305 PRP
English 101	3	8:00-9:00	102 SJT

R2(room/bldg, campus)

campus	room/bldg
East	212 PRP
East	305PRP
West	102 SJT

Given a relation R(A, B, C, D) with the following functional dependencies:

1. $A \rightarrow B, C$
2. $C \rightarrow D$

Determine if the relation R is in Boyce-Codd Normal Form (BCNF). If it is not, explain why and decompose it into BCNF.

Steps to Answer:

1. **Identify Candidate Keys:**
    - A candidate key is an attribute or a combination of attributes that can uniquely identify a tuple in the relation.
    - Identify the candidate key(s) by examining the given functional dependencies.
2. **Check the BCNF Condition:**
    - A relation is in BCNF if, for every functional dependency $X \rightarrow Y$, the attribute set X is a superkey (i.e., it is either a candidate key or it contains a candidate key).
    - Check each functional dependency in the relation to see if the left-hand side (X) is a superkey.
3. **Determine if R is in BCNF:**
    - If all functional dependencies satisfy the BCNF condition, the relation is in BCNF.
    - If any functional dependency does not satisfy this condition, the relation is not in BCNF.
4. **Decompose the Relation (if needed):**
    - If the relation is not in BCNF, decompose it into smaller relations that are in BCNF.
    - Ensure that the decomposition is lossless and preserves all functional dependencies.

1. **Identify Candidate Keys:**
   - From **A → B, C**, A determines B and C, but we also need D.
   - Since **C → D**, A determines C, and C determines D, so **A** can determine **B, C,** and **D**.
   - Therefore, **A** is a candidate key.
2. **Check the BCNF Condition:**
   - For **A → B, C**: **A** is a candidate key, so this functional dependency satisfies BCNF.
   - For **C → D**: **C** is not a candidate key, and it does not contain the candidate key **A**. Therefore, this functional dependency violates the BCNF condition.
3. **Conclusion:**
   - The relation **R(A, B, C, D)** is **not in BCNF** because the functional dependency **C → D** violates the BCNF condition.
4. **Decompose the Relation:**
   - Decompose **R** into two relations:
     1. **R1(C, D)** where **C** is the primary key.
     2. **R2(A, B, C)** where **A** is the primary key.
   - Both **R1** and **R2** are now in BCNF.

By following these steps, you can determine whether the given relation is in BCNF and take the necessary steps to normalize it if it is not.

Consider the following relation schema:
R(A,B,C,D)
The functional dependencies (FDs) for R are:
{A→B,C→D,AB→C}
Is the relation R in Boyce-Codd Normal Form (BCNF)? If not, decompose it into BCNF

Step 1: Check for BCNF violation

A relation R is in BCNF if, for every non-trivial functional dependency $X \to Y$, X is a superkey (i.e., X should contain a candidate key).

- Candidate keys for R can be determined by checking if any combination of attributes determines all other attributes.
  - $A \to B$ is given, but A does not determine C or D, so A alone is not a candidate key.
  - $C \to D$ is given, but C does not determine A or B, so C alone is not a candidate key.
  - $AB \to C$ is given, and by transitivity and augmentation:
    - $AB \to C$ and $C \to D$, so $AB \to CD$.
    - $AB \to C$ and $A \to B$, so $A \to B$, meaning AB determines all attributes (AB is a candidate key).

Check each FD:

- $A \rightarrow B$: A is not a superkey (it doesn't determine C and D), so this violates BCNF.
- $C \rightarrow D$: C is not a superkey (it doesn't determine A and B), so this also violates BCNF.
- $AB \rightarrow C$: AB is a candidate key, so this satisfies BCNF.

Step 2: Decomposition into BCNF

Since $A \rightarrow B$ violates BCNF, decompose R into two relations based on this FD:

1. $R1(A, B)$ with $A \rightarrow B$.
2. $R2(A, C, D)$ with the remaining FDs: $C \rightarrow D$ and $A \rightarrow C$ (from $AB \rightarrow C$).

Check the new relations:

- $R1(A, B)$: A is a candidate key, and $A \rightarrow B$ satisfies BCNF.
- $R2(A, C, D)$: A is a candidate key for this relation, and $A \rightarrow CD$ (since $C \rightarrow D$), which satisfies BCNF.

So, the final relations $R1(A, B)$ and $R2(A, C, D)$ are in BCNF.

Find the highest Normal form of the given Relation R using the given FDs

Let **R={A,B,C,D,E } FD ={ A->BCDE , BC-> ACE and D->E }**

1. CHECK R is in BCNF
2. Compute the candidate keys

{ A and BC are the candidate keys} Prime attributes { A, B, C}

Apply the definition of BCNF

	A->BCDE	, BC-> ACE	and D->E	
BCNF	√	√	X	(NOT IN BCNF)
3NF	√	√	X	(NOT IN 3NF)
2NF	√	√	√	(IN 2NF)

Thank You

BACSE202 - Database Systems

Normalization: First, Second and Third Normal Forms - Boyce Codd Normal Form, Multi-Valued Dependency and Fourth Normal Form - Join Dependency and Fifth Normal Form

Dr. Somasundaram R

Assistant Professor Sr. Grade 2

Emp Id: 23770

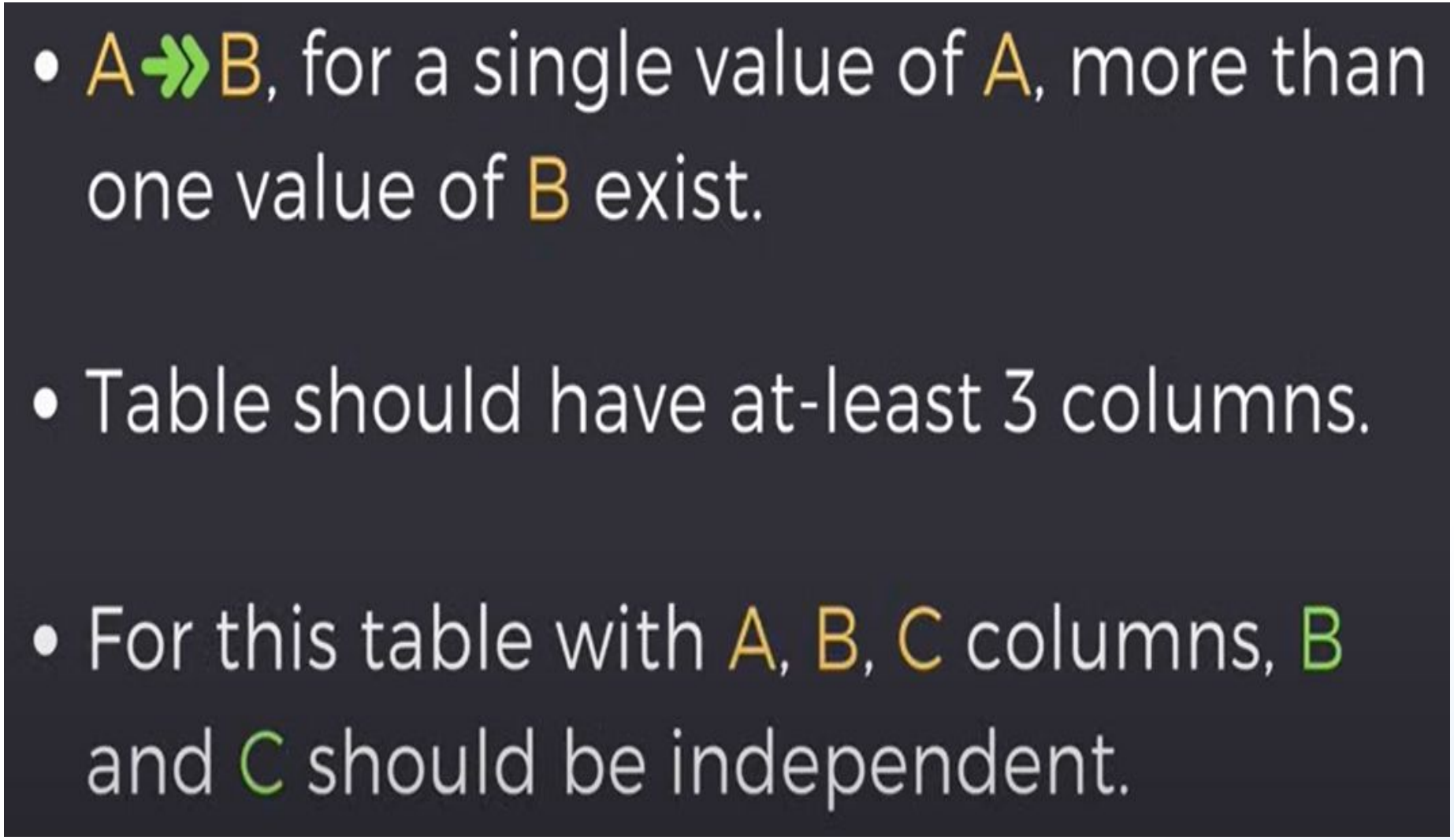
Department of IoT

School of Computer Science and Engineering (SCOPE)

4th Normal Form

A Relation R is said to be in fourth normal form (4NF) if and only if it is in BCNF and contains no more than one multi-valued dependency.

Three conditions for Multivalued Dependency



If all the three conditions are true then we can say that the table may have multivalued dependency

Lets consider an example

A	B	C
A1	B1 B2	C1 C2

We observe that there exist a multivalued dependency in the above table

A->>B and A->>C

Enrollment Table

s_id	course	hobby
1	Science	Cricket
1	Maths	Hockey
2	C#	Cricket
2	Php	Hockey

s_id	course	hobby
1	Science	Cricket
1	Maths	Hockey

s_id	course	hobby
1	Science	Cricket
1	Maths	Hockey
1	Science	Hockey
1	Maths	Cricket

No relationship between **Course and hobby which leads to a BAD DESIGN**

CourseOpted TABLE

s_id	course
1	Science
1	Maths
2	C#
2	Php

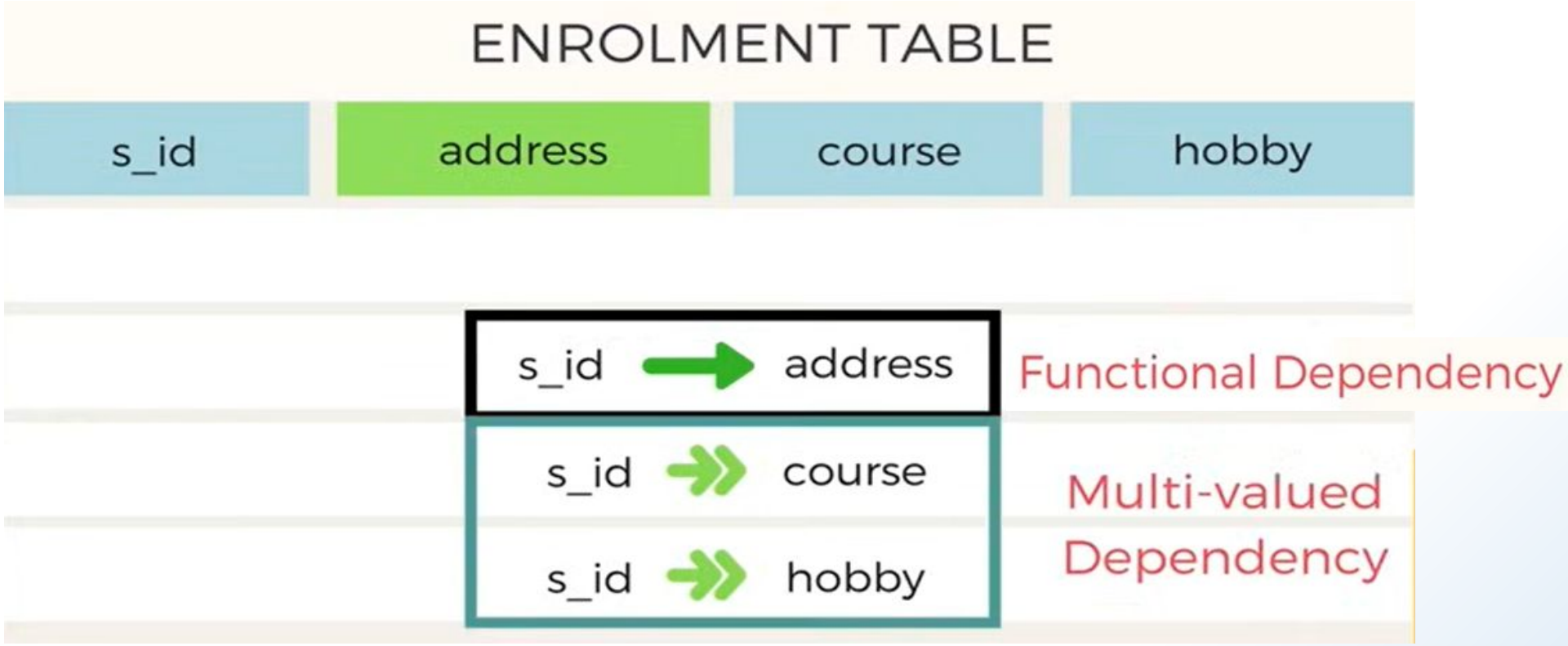
Hobbies TABLE

s_id	hobby
1	Cricket
1	Hockey
2	Cricket
2	Hockey

The Enrollment table is decomposed into two independent tables namely **CourseOpted** and **Hobbies** to make the relation into 4th NF.

Note : A Table may have both the functional dependency and Multivalued dependency

If we add a column address to the Enrollment Table then the table has the dependencies as follows :



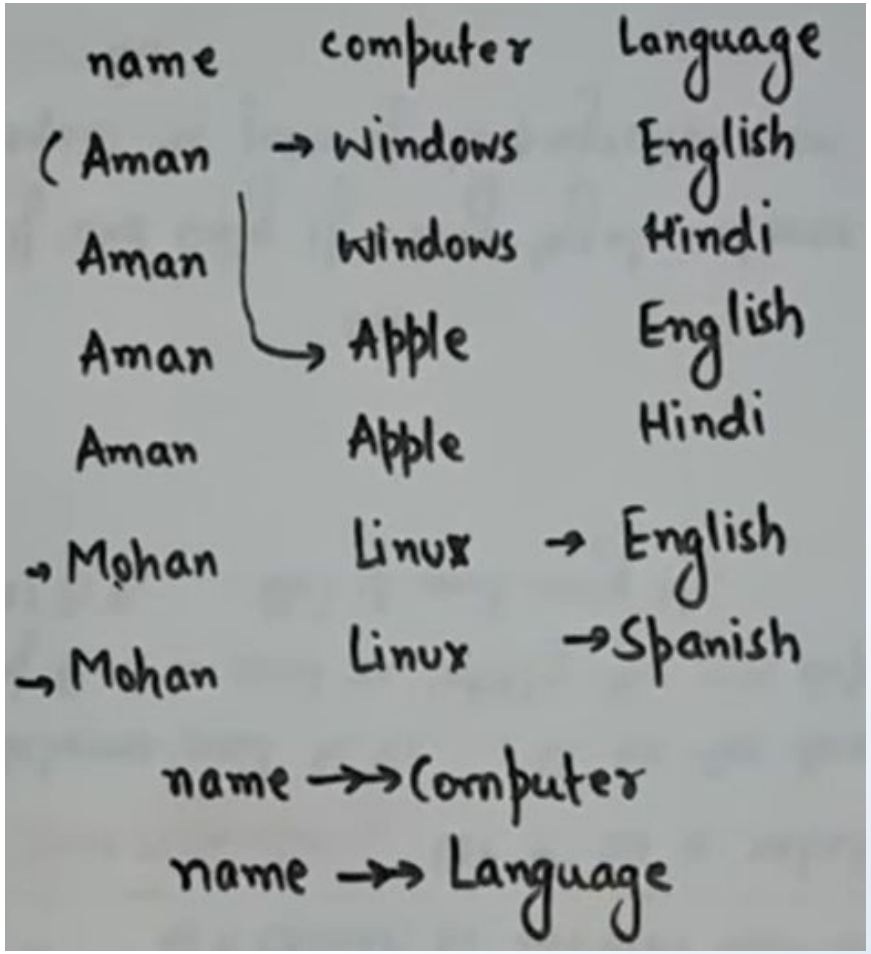
Now the given relation R is decomposed into 3 Relations

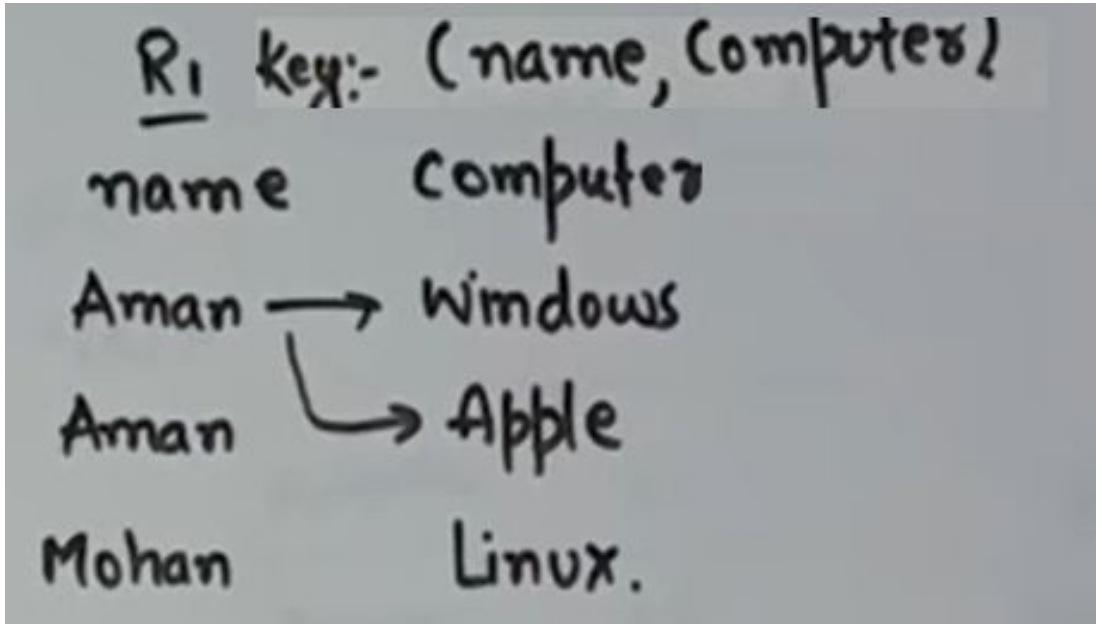
(SID & COURSE), (SID & HOBBY) and (SID & ADDRESS)

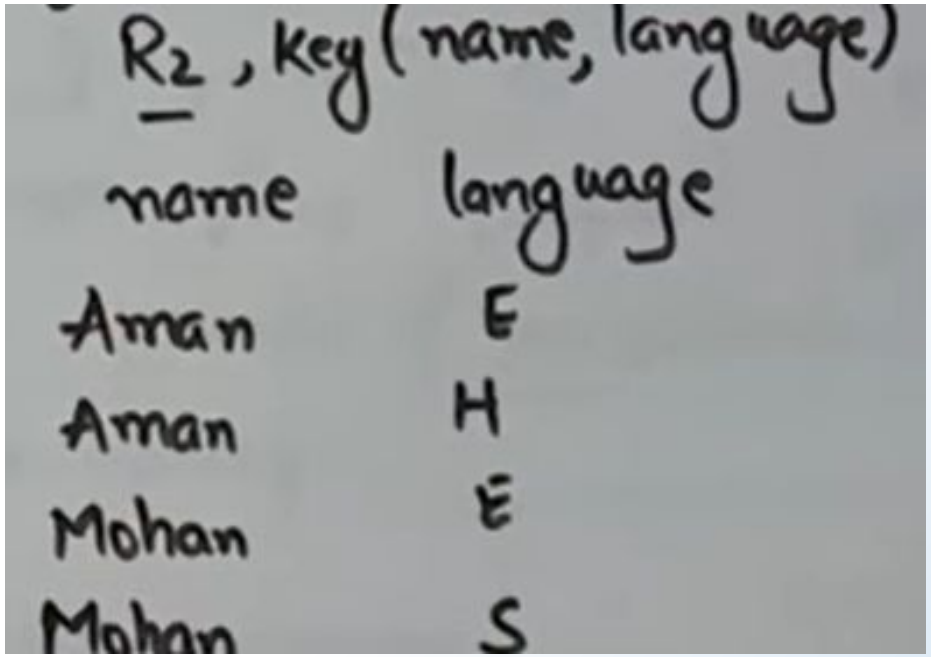
Consider the following table with the following records. Normalize this table. Is this table is in 4NF ? IF not decompose it to 4NF

NAME	COMPUTER-OS	LANGUAGE
AMAN	WINDOWS APPLE	ENGLISH HINDI
MOHAN	LINUX	ENGLISH SPANISH

name	computer	Language
Aman	Windows	English
Aman	Windows	Hindi
Aman	Apple	English
Aman	Apple	Hindi
Mohan	Linux	English
Mohan	Linux	Spanish




R1 key:- (name, Computer)
name
Computer
Aman
Windows
Aman
Apple
Mohan
Linux.


R2, key (name, language)
name
language
Aman
E
Aman
H
Mohan
E
Mohan
S

5th Normal form :

A table is in fifth normal form (5NF) or Project-Join Normal Form (PJNF) if it is in 4NF and it cannot have a lossless decomposition into any number of smaller table.

Fifth Normal Form(5NF) :- A relation 'R' is in 5th NF if and only if the following Cond^n are Satisfied Simultaneously:-

i) R is already in 4NF (No M.V.D in R)

ii) It Cannot be further non-loss decomposed.

JOIN Dependency: Let 'R' be a relation Schema and $R_1, R_2, \ldots R_n$ be the decompositio of R. R is said to satisfy the join dependency $*(R_1, R_2, \ldots R_n)$ if and only if:

$$\Pi_{R_1}(R) \bowtie \Pi_{R_2}(R) \bowtie \ldots\ldots \bowtie \Pi_{R_n}(R) = R$$

(order of join doesn't matter)

OR

if and only if every legal instance $r(R)$ is equal to join of its projections on $R_1, R_2, \ldots R_n$.

Example of Join Dependency:

Agent	Company	Product
Aman	C1	Pen drive
Aman	C1	MIC
Aman	C2	Speaker
Mohan	C1	Speaker

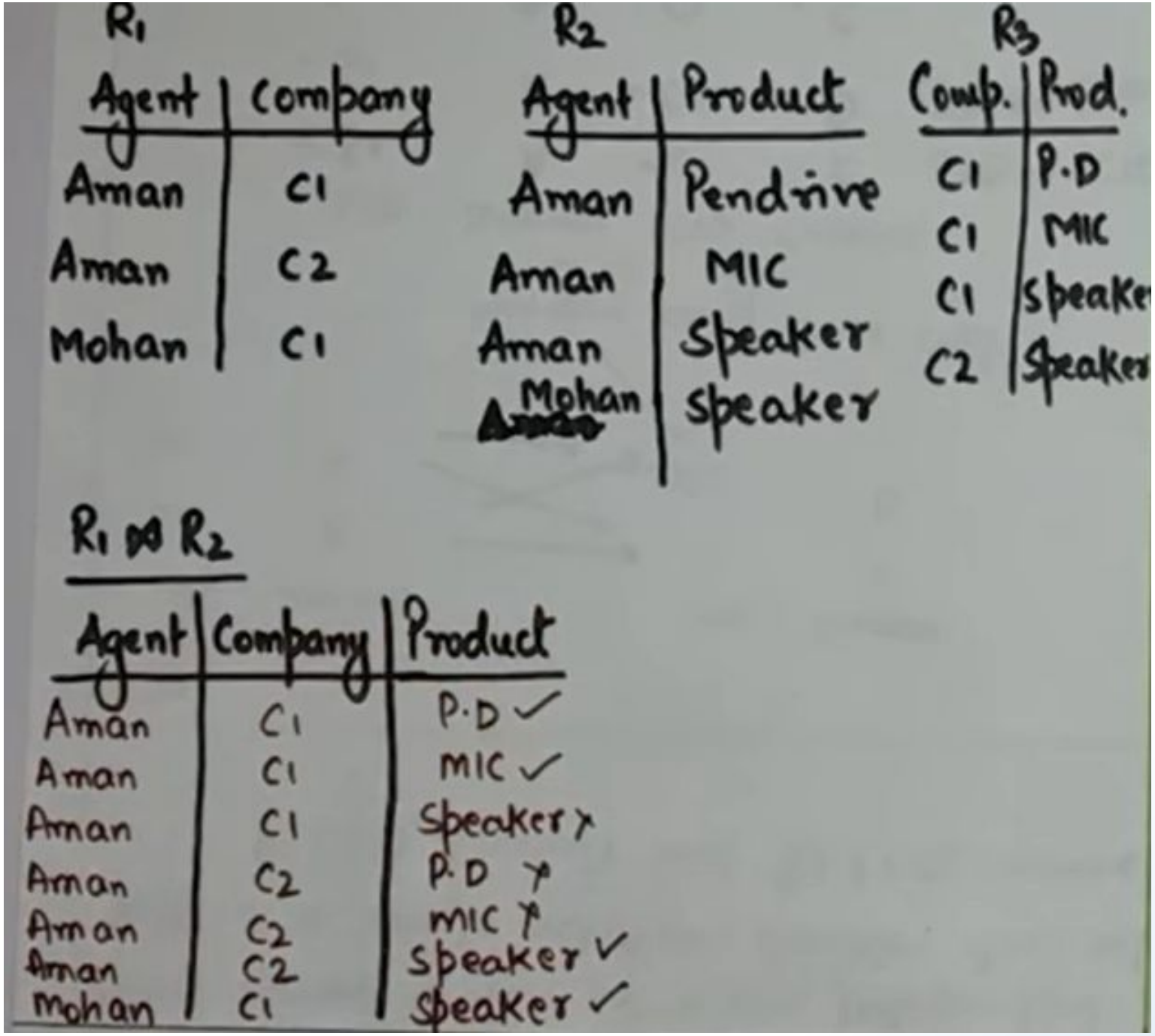
R_1 (Agent | Company)

R_2 (Agent | Product)

R_3 (Company | Product)

$R_1 \bowtie R_2 = R$

$R_1 \bowtie R_2 \bowtie R_3 = R$ (J.D)



R_1

Agent	Company
Aman	C1
Aman	C2
Mohan	C1

R_2

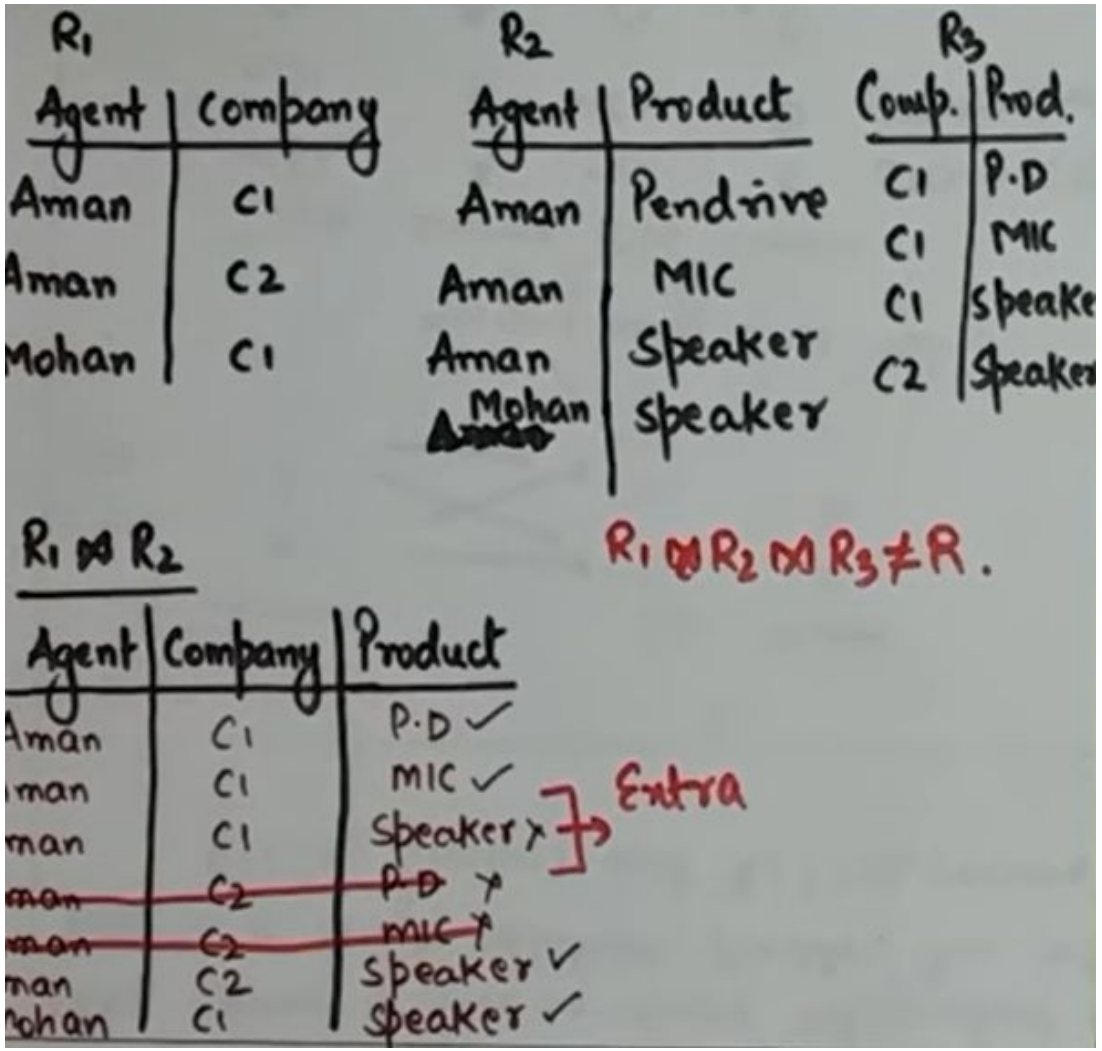
Agent	Product
Aman	Pendrive
Aman	MIC
Aman	Speaker
Mohan	speaker

R_3

Comp.	Prod.
C1	P.D
C1	MIC
C1	Speaker
C2	Speaker

$R_1 \bowtie R_2$

Agent	Company	Product
Aman	C1	P.D ✓
Aman	C1	MIC ✓
Aman	C1	Speaker ✗
Aman	C2	P.D ✗
Aman	C2	MIC ✗
Aman	C2	Speaker ✓
Mohan	C1	Speaker ✓



Therefore there is NO Join DEPENDENCY EXISTS

JOIN Dependency Rule :- Holds Good only if a table can be retransformed back without any loss of infoⁿ from the join of certain specified projections on it.
[Find JDs in table and if it exits then table is not in 5NF]

A relation R is in 5NF w.r.t a set of F of functional, multivalued and join dependencies if for every non-trivial join dependency *(R1, ... Rn) in F^+, every R_i is a Superkey of R.

Ques) Is the table in 5NF?

PName	Skill	Job
Aman	DBA	J1
Mohan	Tester	J2
Rohan	Programmer	J3
Sohan	Analyst	J1

R_1(Pname, Skill)

Pname	Skill
Aman	DBA
Mohan	Tester
Rohan	Programmer
Sohan	Analyst

R_2(Pname, job)

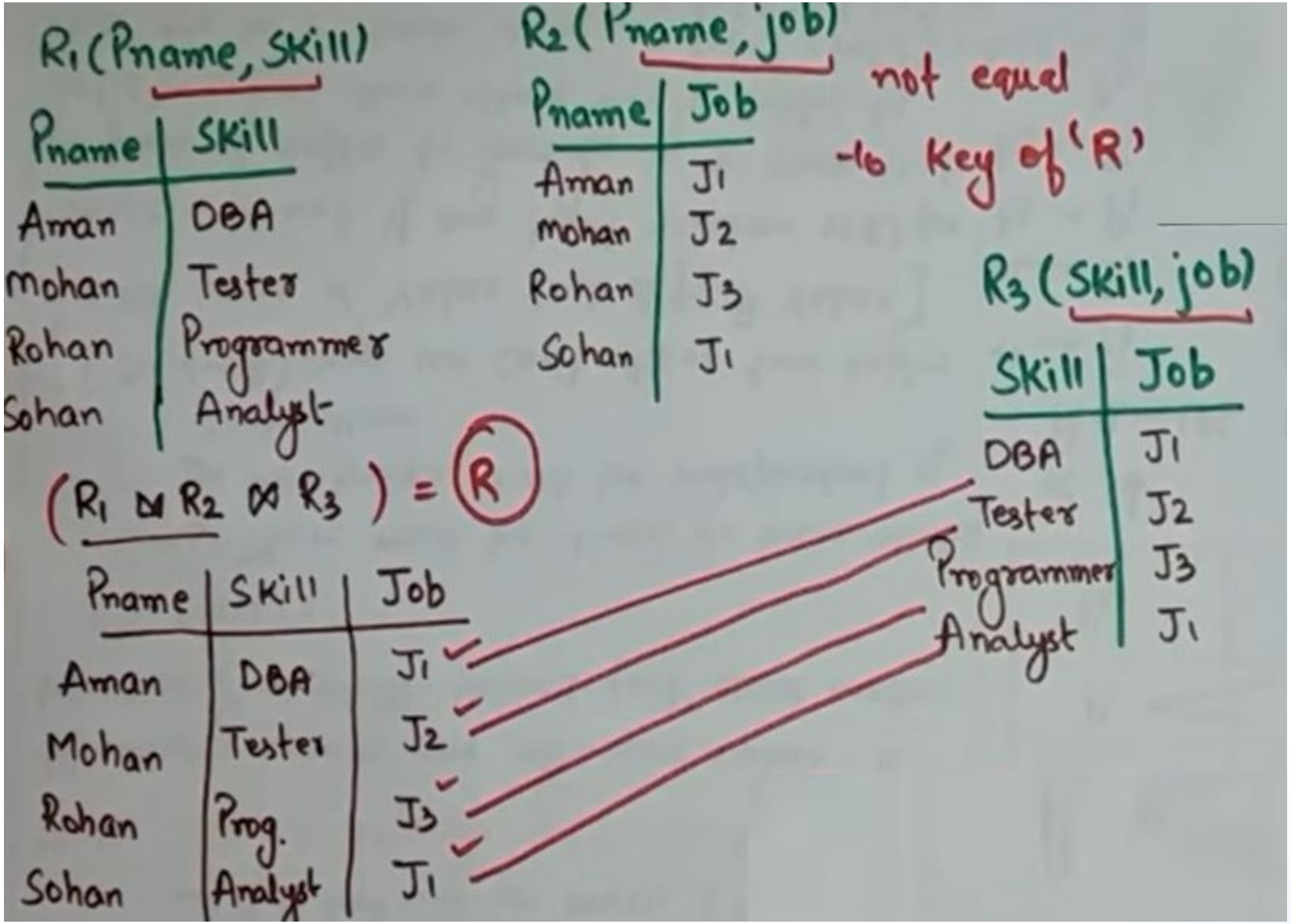
Pname	Job
Aman	J1
mohan	J2
Rohan	J3
Sohan	J1

R_3(Skill, job)

Skill	Job
DBA	J1
Tester	J2
Programmer	J3
Analyst	J1

$R_1 \bowtie R_2$

Pname	Skill	Job
Aman	DBA	J1
Mohan	Tester	J2
Rohan	Prog.	J3
Sohan	Analyst	J1



Therefore the given relation R is in 5NF

Thank You